光影资产项目技术文档

----- **作者**：赵杰 徐爱姣 ------

目录

1. 项目整体介绍
2. 项目结构解析
3. 后端技术详解
4. 前端技术详解
5. 核心业务流程
6. 如何运行项目
7. 常见问题解答

---

1. 项目整体介绍

1.1 这个项目是做什么的?

简单来说：这是一个设备借用管理系统，就像图书馆借书系统一样，但我们管理的是摄影设备。

使用场景：

- 学生想借用学校的相机拍摄作品 → 在系统里提交申请
- 管理员收到申请 → 在系统里批准或拒绝
- 学生使用完毕归还设备 → 管理员在系统里确认归还

1.2 为什么要做这个项目?

传统方式的问题：

```
学生 → 填写纸质表格 → 管理员手工登记 → 设备借出
       ↓
容易出错、查询不便、状态不明
```

我们的解决方案：

```
学生 → 网页提交申请 → 管理员在线审批 → 系统自动记录
       ↓
高效、准确、可追溯
```

1.3 项目的核心功能

用户端（学生使用）

1. **登录系统** - 输入用户名和密码
2. **浏览设备** - 看到所有可借的设备（相机、镜头、灯光等）
3. **申请借用** - 选择设备，填写借用目的和归还日期
4. **查看记录** - 看到自己的借用历史

管理端（老师/管理员使用）

1. **仪表盘** - 看到设备总数、用户总数、待审批数量
2. **设备管理** - 添加、修改、删除设备
3. **用户管理** - 添加、修改、删除用户
4. **借用审批** - 批准或拒绝学生的借用申请
5. **归还确认** - 学生归还后，管理员确认

---

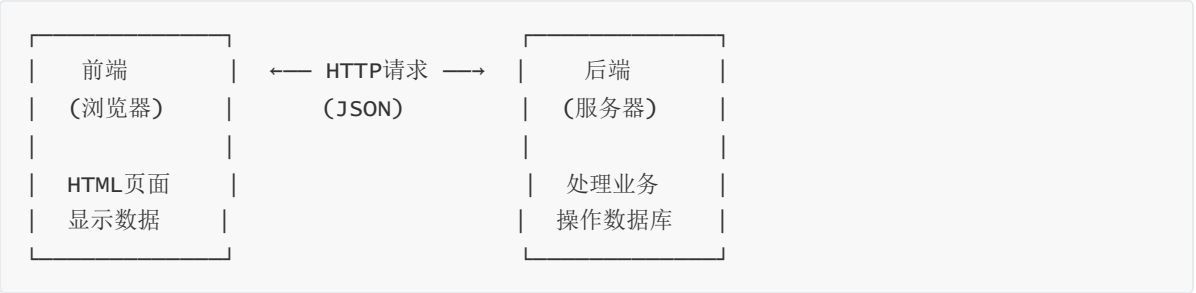
2. 项目结构解析

2.1 项目文件夹结构

```
project/
├── DeviceManage-backend/          ← 后端代码（Java）
│   ├── src/
│   │   ├── main/
│   │   │   ├── java/              ← Java源代码
│   │   │   └── resources/         ← 配置文件
│   │   └── test/                  ← 测试代码
│   └── pom.xml                    ← Maven配置文件
│
├── DeviceManage-frontend/         ← 前端代码（HTML/CSS/JS）
│   ├── index.html                 ← 登录页面
│   ├── admin/                     ← 管理员页面
│   │   ├── dashboard.html         ← 仪表盘
│   │   ├── devices.html           ← 设备管理
│   │   ├── users.html             ← 用户管理
│   │   └── borrow-approval.html   ← 借用审批
│   ├── user/                      ← 用户页面
│   │   ├── devices.html           ← 设备浏览
│   │   └── borrow-records.html    ← 我的借用记录
│   ├── css/                       ← 样式文件
│   ├── js/                        ← JavaScript文件
│   └── static/                    ← 图片资源
│
└── docs/                          ← 文档
    ├── 接口文档.md
    └── device_manage.sql          ← 数据库脚本
```

2.2 前端和后端是如何配合的?

前后端分离架构:



举个例子:

1. 用户在浏览器打开设备列表页面（前端）
2. 页面加载后，JavaScript发送请求到后端：`GET /devices`
3. 后端收到请求，从数据库查询设备数据
4. 后端返回JSON格式的数据给前端
5. 前端JavaScript接收数据，动态生成HTML显示在页面上

为什么要分离?

- 前端专注界面展示，后端专注业务逻辑
- 可以独立开发、独立部署
- 前端可以换成手机App，后端不用改

---

3. 后端技术详解

3.1 后端用到的技术（从基础到高级）

技术1：Spring Boot（核心框架）

是什么?

- Spring Boot是一个Java框架，帮我们快速搭建Web应用
- 就像盖房子，Spring Boot提供了地基、框架和基础设施

为什么用它?

- 不用写很多配置代码，开箱即用
- 内置了Web服务器（Tomcat），不用单独安装
- 集成了很多常用功能

怎么用的?

```java
@SpringBootApplication  // 这个注解表示这是Spring Boot应用的入口
public class DeviceManageBackendApplication {
    public static void main(String[] args) {
        SpringApplication.run(DeviceManageBackendApplication.class, args);
        // 运行这个main方法，整个后端服务就启动了
    }
}
```

技术2：MyBatis Plus（数据库操作）

是什么?

- MyBatis Plus是一个数据库操作工具
- 帮我们把Java对象和数据库表对应起来

传统方式 vs MyBatis Plus:

传统JDBC方式（复杂）：

```java
// 需要写很多代码
Connection conn = DriverManager.getConnection(url, user, password);
String sql = "SELECT * FROM users WHERE id = ?";
PreparedStatement stmt = conn.prepareStatement(sql);
stmt.setLong(1, userId);
ResultSet rs = stmt.executeQuery();
if (rs.next()) {
    User user = new User();
    user.setId(rs.getLong("id"));
    user.setUsername(rs.getString("username"));
    // ... 还要一个个字段设置
}
// 还要关闭连接、处理异常...
```

MyBatis Plus方式（简单）：

```java
// 只需要一行代码!
User user = userMapper.selectById(userId);
```

怎么实现的?

步骤1：定义实体类（Entity）

```java
@Data  // Lombok注解，自动生成getter/setter
@TableName("users")  // 对应数据库的users表
public class User {
    @TableId(type = IdType.AUTO)  // 主键，自动增长
    private Long id;

    private String username;  // 对应数据库的username字段
    private String password;
    private String realName;
    private String role;
    private String status;
```

```
    private LocalDateTime createdAt;
    private LocalDateTime updatedAt;
}
```

步骤2：定义Mapper接口

```
// 继承BaseMapper，就自动有了增删改查方法
public interface UserMapper extends BaseMapper<User> {
    // 不用写任何代码，就已经有了：
    // - selectById(id)       查询单个
    // - selectList(wrapper) 查询列表
    // - insert(user)         插入
    // - updateById(user)     更新
    // - deleteById(id)       删除
}
```

步骤3：在Service中使用

```
@Service
public class UserServiceImpl implements UserService {
    @Autowired  // 自动注入Mapper
    private UserMapper userMapper;

    public User getUserById(Long id) {
        return userMapper.selectById(id);  // 直接调用
    }
}
```

技术3：Spring Security + JWT（安全认证）

是什么?

- Spring Security：安全框架，负责认证（你是谁）和授权（你能干什么）
- JWT：一种Token格式，用于在前后端之间传递用户信息

为什么需要认证?

- 防止未登录用户访问系统
- 防止普通用户访问管理员功能
- 保护用户数据安全

JWT的工作原理（重点理解）：

```
┌──────────────────────────────────────────────────┐
│  第1步：用户登录                                  │
├──────────────────────────────────────────────────┤
│  用户输入：username=admin, password=123456        │
│  后端验证：查数据库，密码正确 ✓                    │
│  后端生成Token：eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...|
│  Token内容：{userId: 1, username: "admin", role: "ADMIN"}|
│  返回给前端：{token: "...", userInfo: {...}}      │
└──────────────────────────────────────────────────┘
```

```
┌─────────────────────────────────────────────────────┐
│  第2步：前端保存Token                                 │
├─────────────────────────────────────────────────────┤
│  localStorage.setItem('token', token);              │
│  // Token保存在浏览器本地存储中                        │
└─────────────────────────────────────────────────────┘

┌─────────────────────────────────────────────────────┐
│  第3步：后续请求携带Token                             │
├─────────────────────────────────────────────────────┤
│  前端发送请求：GET /devices                           │
│  请求头：Authorization: Bearer eyJhbGci...          │
│  // 每次请求都带上Token                               │
└─────────────────────────────────────────────────────┘

┌─────────────────────────────────────────────────────┐
│  第4步：后端验证Token                                 │
├─────────────────────────────────────────────────────┤
│  后端接收请求 → 提取Token → 验证签名 → 解析内容        │
│  如果Token有效：继续处理请求                          │
│  如果Token无效：返回401错误，要求重新登录             │
└─────────────────────────────────────────────────────┘
```

代码实现：

```java
// 1. 生成Token（登录时）
public String generateToken(Long userId, String username, String role) {
    Map<String, Object> claims = new HashMap<>();
    claims.put("userId", userId);      // 放入用户ID
    claims.put("username", username);  // 放入用户名
    claims.put("role", role);          // 放入角色

    Date now = new Date();
    Date expiryDate = new Date(now.getTime() + 86400000);  // 24小时后过期

    return Jwts.builder()
            .claims(claims)            // 设置内容
            .issuedAt(now)             // 签发时间
            .expiration(expiryDate)    // 过期时间
            .signWith(key)             // 用密钥签名
            .compact();                // 生成Token字符串
}

// 2. 验证Token（每次请求时）
public Boolean validateToken(String token) {
    try {
        Claims claims = getClaimsFromToken(token);  // 解析Token
        return !claims.getExpiration().before(new Date());  // 检查是否过期
    } catch (Exception e) {
        return false;  // Token无效
    }
}
```

密码加密：

```
// 为什么要加密？
// 如果数据库被黑客攻击，密码明文存储会导致所有用户账号泄露

// 注册/创建用户时：加密密码
String rawPassword = "123456";  // 用户输入的原始密码
String encodedPassword = passwordEncoder.encode(rawPassword);
// 加密后：$2a$10$b68xPCyu2uuxoo9tVV/XM.HB4vd5EksRJl2E8rTY8g.i7o3TnD3XW
user.setPassword(encodedPassword);  // 存储加密后的密码

// 登录时：验证密码
boolean isMatch = passwordEncoder.matches(rawPassword, encodedPassword);
if (isMatch) {
    // 密码正确，允许登录
} else {
    // 密码错误，拒绝登录
}
```

BCrypt的特点：

- 单向加密，无法解密（只能验证是否匹配）
- 每次加密结果不同（但都能验证通过）
- 即使数据库泄露，黑客也无法得到原始密码

3.2 后端代码结构（分层架构）

我们的后端代码分为几层，每层负责不同的事情：

```
┌─────────────────────────────────────────────────┐
│  Controller层（控制器）                          │
│  作用：接收前端请求，调用Service，返回响应        │
│  文件位置：controller/                           │
├─────────────────────────────────────────────────┤
│  Service层（业务逻辑）                           │
│  作用：处理业务逻辑，比如验证、计算、调用Mapper   │
│  文件位置：service/ 和 service/impl/             │
├─────────────────────────────────────────────────┤
│  Mapper层（数据访问）                            │
│  作用：操作数据库，执行SQL语句                   │
│  文件位置：mapper/ 和 resources/mapper/          │
├─────────────────────────────────────────────────┤
│  Entity层（实体类）                              │
│  作用：对应数据库表，一个类对应一张表            │
│  文件位置：entity/                               │
├─────────────────────────────────────────────────┤
│  DTO/VO层（数据传输对象）                        │
│  作用：前后端交互的数据格式                      │
│  文件位置：dto/ 和 vo/                           │
└─────────────────────────────────────────────────┘
```

举个例子：查询用户列表的完整流程

第1步：前端发送请求

```
// 前端JavaScript代码
fetch('http://localhost:9011/admin/users?page=1&size=10', {
    method: 'GET',
    headers: {
        'Authorization': 'Bearer ' + token  // 携带Token
    }
})
```

第2步：Controller接收请求

```
// 文件：AdminUserController.java
@RestController  // 表示这是一个控制器，处理HTTP请求
@RequestMapping("/admin/users")  // 这个类处理 /admin/users 开头的请求
public class AdminUserController {

    @Autowired  // 自动注入UserService
    private UserService userService;

    // 处理 GET /admin/users 请求
    @GetMapping
    public Result<PageResult<User>> getUserList(
            @RequestParam(required = false) String username,  // 查询参数：用户名
            @RequestParam(required = false) String role,      // 查询参数：角色
            @RequestParam(defaultValue = "1") Integer page,   // 查询参数：页码，默
认1
            @RequestParam(defaultValue = "10") Integer size   // 查询参数：每页数
量，默认10
    ) {
        // 调用Service层的方法
        PageResult<User> result = userService.getUserList(username, role, page,
size);

        // 返回统一格式的响应
        return Result.success("获取成功", result);
    }
}
```

理解要点：

- `@RestController`：告诉Spring这是一个控制器
- `@RequestMapping`：定义这个类处理哪些URL
- `@GetMapping`：处理GET请求（查询数据）
- `@PostMapping`：处理POST请求（创建数据）
- `@PutMapping`：处理PUT请求（更新数据）
- `@DeleteMapping`：处理DELETE请求（删除数据）
- `@RequestParam`：获取URL中的查询参数，比如 `?page=1&size=10`
- `@PathVariable`：获取URL路径中的变量，比如 `/users/{id}`

第3步：Service处理业务逻辑

```
// 文件：UserServiceImpl.java
@Service  // 表示这是一个服务类
public class UserServiceImpl implements UserService {

    @Autowired  // 自动注入Mapper
    private UserMapper userMapper;

    @Override
    public PageResult<User> getUserList(String username, String role,
                                        Integer page, Integer size) {
        // 1. 计算偏移量（分页用）
        Integer offset = (page - 1) * size;
        // 比如：第1页，offset=0；第2页，offset=10

        // 2. 调用Mapper查询数据
        List<User> list = userMapper.selectUserList(username, role, offset,
size);

        // 3. 查询总数
        Long total = userMapper.countUsers(username, role);

        // 4. 封装成分页结果
        return new PageResult<>(total, page, size, list);
    }
}
```

理解要点：

- Service层负责业务逻辑，比如计算、验证、调用Mapper
- 不直接操作数据库，而是调用Mapper
- 可以调用多个Mapper，组合复杂的业务逻辑

第4步：Mapper操作数据库

```
// 文件：UserMapper.java（接口）
public interface UserMapper extends BaseMapper<User> {
    // 自定义查询方法
    List<User> selectUserList(@Param("username") String username,
                              @Param("role") String role,
                              @Param("offset") Integer offset,
                              @Param("size") Integer size);

    Long countUsers(@Param("username") String username,
                    @Param("role") String role);
}
```

```
<!-- 文件：UserMapper.xml（SQL语句） -->
<mapper namespace="chat.hbcit.devicemanagebackend.mapper.UserMapper">

    <!-- 查询用户列表 -->
    <select id="selectUserList"
resultType="chat.hbcit.devicemanagebackend.entity.User">
```

```
        SELECT id, username, password, real_name, role, status, created_at,
updated_at
        FROM users
        <where>
            <!-- 如果username不为空，添加模糊查询条件 -->
            <if test="username != null and username != ''">
                AND username LIKE CONCAT('%', #{username}, '%')
            </if>
            <!-- 如果role不为空，添加精确查询条件 -->
            <if test="role != null and role != ''">
                AND role = #{role}
            </if>
        </where>
        ORDER BY created_at ASC
        LIMIT #{offset}, #{size}
    </select>

    <!-- 统计用户总数 -->
    <select id="countUsers" resultType="java.lang.Long">
        SELECT COUNT(*)
        FROM users
        <where>
            <if test="username != null and username != ''">
                AND username LIKE CONCAT('%', #{username}, '%')
            </if>
            <if test="role != null and role != ''">
                AND role = #{role}
            </if>
        </where>
    </select>

</mapper>
```

理解要点：

- Mapper接口定义方法名
- XML文件写SQL语句
- `#{username}` 是占位符，会自动替换成参数值
- `<if>` 标签实现动态SQL，根据条件决定是否添加WHERE条件

第5步：返回结果给前端

```
// Controller返回的Result对象会自动转换成JSON
{
    "code": 200,
    "message": "获取成功",
    "data": {
        "total": 100,
        "page": 1,
        "size": 10,
        "list": [
            {"id": 1, "username": "admin", "realName": "管理员", ...},
            {"id": 2, "username": "user001", "realName": "张三", ...}
        ]
```

```
    }
}
```

3.3 核心概念详解

概念1：依赖注入（@Autowired）

是什么?

- 不用自己new对象，Spring自动帮你创建并注入

传统方式:

```
public class UserServiceImpl {
    // 需要自己创建Mapper对象
    private UserMapper userMapper = new UserMapperImpl();
}
```

Spring方式:

```
public class UserServiceImpl {
    @Autowired  // Spring自动注入
    private UserMapper userMapper;
    // 不用管怎么创建，Spring会自动处理
}
```

为什么这样做?

- 解耦：不需要关心对象如何创建
- 方便测试：可以注入Mock对象
- 统一管理：Spring容器管理所有对象

概念2：RESTful API

是什么?

- 一种设计风格，用HTTP方法表示操作类型

四种HTTP方法:

```
GET    → 查询数据（不改变数据）
POST   → 创建数据（新增）
PUT    → 更新数据（修改）
DELETE → 删除数据
```

URL设计规范:

```
GET    /users          → 查询所有用户
GET    /users/1        → 查询ID=1的用户
POST   /users          → 创建新用户
PUT    /users/1        → 更新ID=1的用户
DELETE /users/1        → 删除ID=1的用户
```

我们项目中的例子：

```
@RestController
@RequestMapping("/admin/users")
public class AdminUserController {

    @GetMapping              // GET /admin/users
    public Result<PageResult<User>> getUserList(...) { }

    @GetMapping("/{id}")     // GET /admin/users/1
    public Result<User> getUserById(@PathVariable Long id) { }

    @PostMapping             // POST /admin/users
    public Result<User> createUser(@RequestBody User user) { }

    @PutMapping("/{id}")     // PUT /admin/users/1
    public Result<?> updateUser(@PathVariable Long id, @RequestBody User user) {
}

    @DeleteMapping("/{id}")  // DELETE /admin/users/1
    public Result<?> deleteUser(@PathVariable Long id) { }
}
```

概念3：统一响应格式（Result）

为什么需要统一格式?

- 前端可以用同样的方式处理所有响应
- 便于错误处理
- 代码更规范

Result类的设计：

```
public class Result<T> {
    private Integer code;       // 状态码：200成功，400/401/403/500错误
    private String message;     // 消息：给用户看的提示
    private T data;             // 数据：查询结果、创建的对象等

    // 成功响应（带数据）
    public static <T> Result<T> success(T data) {
        return new Result<>(200, "操作成功", data);
    }

    // 成功响应（自定义消息）
    public static <T> Result<T> success(String message, T data) {
        return new Result<>(200, message, data);
    }

    // 失败响应
    public static <T> Result<T> error(Integer code, String message) {
        return new Result<>(code, message, null);
    }
}
```

使用示例：

```
// 成功的情况
return Result.success("获取成功", userList);
// 返回：{"code": 200, "message": "获取成功", "data": [...]}

// 失败的情况
return Result.error(404, "用户不存在");
// 返回：{"code": 404, "message": "用户不存在", "data": null}
```

概念4：异常处理

为什么需要异常处理?

- 程序运行中可能出现各种错误
- 需要统一处理，返回友好的错误提示

我们的实现：

步骤1：定义业务异常类

```
// 文件：BusinessException.java
public class BusinessException extends RuntimeException {
    private Integer code;  // 错误码

    public BusinessException(String message) {
        super(message);
        this.code = 500;
    }

    public BusinessException(Integer code, String message) {
        super(message);
        this.code = code;
    }
}
```

步骤2：在业务代码中抛出异常

```
public User getUserById(Long id) {
    User user = userMapper.selectById(id);
    if (user == null) {
        // 抛出业务异常
        throw new BusinessException(404, "用户不存在");
    }
    return user;
}
```

步骤3：全局异常处理器统一捕获

```
// 文件：GlobalExceptionHandler.java
@RestControllerAdvice  // 全局异常处理器
public class GlobalExceptionHandler {
```

```
    // 捕获业务异常
    @ExceptionHandler(BusinessException.class)
    public Result<?> handleBusinessException(BusinessException e) {
        // 返回统一格式的错误响应
        return Result.error(e.getCode(), e.getMessage());
    }

    // 捕获参数校验异常
    @ExceptionHandler(MethodArgumentNotValidException.class)
    public Result<?> handleValidationException(MethodArgumentNotValidException e)
{
        String message = e.getBindingResult().getFieldErrors().stream()
                .map(FieldError::getDefaultMessage)
                .collect(Collectors.joining(", "));
        return Result.error(400, message);
    }

    // 捕获所有其他异常
    @ExceptionHandler(Exception.class)
    public Result<?> handleException(Exception e) {
        logger.error("系统异常", e);
        return Result.error(500, "系统内部错误");
    }
}
```

好处:

- 不需要在每个方法里写try-catch
- 统一的错误响应格式
- 避免敏感信息泄露

3.4 数据库表详解

表1：users（用户表）

表结构:

```
CREATE TABLE users (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,  -- 用户ID，自动增长
    username VARCHAR(50) NOT NULL UNIQUE,  -- 用户名，唯一
    password VARCHAR(255) NOT NULL,        -- 密码（加密后）
    real_name VARCHAR(50) NOT NULL,        -- 真实姓名
    role VARCHAR(20) NOT NULL DEFAULT 'USER',  -- 角色：ROOT/ADMIN/USER
    status VARCHAR(20) NOT NULL DEFAULT 'ACTIVE',  -- 状态：ACTIVE/INACTIVE
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  -- 创建时间
    updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP  -- 更新时间
);
```

示例数据:

```
id | username | password                    | real_name | role  | status
---|----------|-----------------------------|-----------|-------|--------
1  | root     | $2a$10$cPZGMzNQthunR0b...      | 超级管理员 | ROOT  | ACTIVE
2  | admin    | $2a$10$b68xPCyu2uuxoo9...      | 管理员     | ADMIN | ACTIVE
10 | 赵杰     | $2a$10$Jp/jfpSbDMlv8rn...      | 赵杰       | USER  | ACTIVE
```

角色权限说明：

- **ROOT**：超级管理员，可以管理所有用户（包括ADMIN）
- **ADMIN**：管理员，可以管理设备、审批借用、管理普通用户
- **USER**：普通用户，只能浏览设备和申请借用

表2：categories（分类表）

表结构：

```sql
CREATE TABLE categories (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(50) NOT NULL UNIQUE,  -- 分类名称，唯一
    description VARCHAR(255),          -- 分类描述
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP
);
```

示例数据：

```
id | name   | description
---|--------|--------------------------
1  | 相机   | 各类相机设备，包括单反、无反
2  | 镜头   | 各类镜头设备
3  | 灯光   | 各类灯光设备
4  | 线材   | 各类线材设备
5  | 三脚架 | 各类三脚架和稳定器设备
```

表3：devices（设备模板表）

表结构：

```sql
CREATE TABLE devices (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
    category_id BIGINT NOT NULL,       -- 分类ID（外键）
    name VARCHAR(100) NOT NULL,        -- 设备名称
    model VARCHAR(100) NOT NULL,       -- 设备型号
    status VARCHAR(20) NOT NULL DEFAULT 'NORMAL',  -- 状态（已废弃，由实例状态决定）
    description TEXT,                  -- 设备描述
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
    FOREIGN KEY (category_id) REFERENCES categories(id)  -- 外键约束
);
```

示例数据:

```
id | category_id | name            | model     | status | description
---|-------------|-----------------|-----------|--------|------------------
1  | 1           | 佳能 EOS R5     | EOS R5    | NORMAL | 全画幅无反相机
2  | 1           | 索尼 A7M4       | ILCE-7M4  | NORMAL | 全画幅无反相机
6  | 2           | 佳能 RF 24-70mm | RF 24-70  | NORMAL | 标准变焦镜头
```

理解要点:

- 这是"设备模板"，描述设备的通用信息
- 一个设备模板可以有多个设备实例
- `category_id` 是外键，关联到categories表

表4：device_instances（设备实例表）★核心创新★

表结构:

```
CREATE TABLE device_instances (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
    device_id BIGINT NOT NULL,          -- 设备ID（外键，关联devices表）
    instance_name VARCHAR(100) NOT NULL,  -- 实例名称，如"佳能EOS R5-001"
    location VARCHAR(100) NOT NULL,     -- 存放位置
    status VARCHAR(20) NOT NULL DEFAULT 'NORMAL',  -- 状态：
NORMAL/MAINTENANCE/SCRAPPED
    description TEXT,                   -- 实例描述
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
    FOREIGN KEY (device_id) REFERENCES devices(id) ON DELETE CASCADE  -- 级联删除
);
```

示例数据:

```
id | device_id | instance_name         | location | status      | description
---|-----------|-----------------------|----------|-------------|-------------
1  | 1         | 佳能 EOS R5-001       | A区-001  | NORMAL      | 状态良好
2  | 2         | 索尼 A7M4-001         | A区-002  | NORMAL      | 状态良好
4  | 6         | 佳能 RF 24-70mm-001   | B区-001  | NORMAL      | 状态良好
```

为什么要设计设备实例表?

场景说明:

```
学校有3台佳能EOS R5相机：
- 第1台（编号001）：正常，放在A区-001，可以借
- 第2台（编号002）：已被学生A借走
- 第3台（编号003）：正在维修，不能借

如果只有devices表，无法区分这3台相机的状态
有了device_instances表，每台相机都是独立的记录，状态独立管理
```

设备模板 vs 设备实例：

```
设备模板（devices）：佳能 EOS R5
    ├─ 实例1（device_instances）：佳能 EOS R5-001（正常）
    ├─ 实例2（device_instances）：佳能 EOS R5-002（借用中）
    └─ 实例3（device_instances）：佳能 EOS R5-003（维修中）
```

借用时的逻辑：

1. 学生申请借用"佳能 EOS R5"（设备模板）
2. 系统查询该设备下的所有实例
3. 筛选出状态为"正常"且未被借用的实例
4. 自动分配第一个可用实例（比如001）
5. 创建借用记录，关联到device_instance_id=1

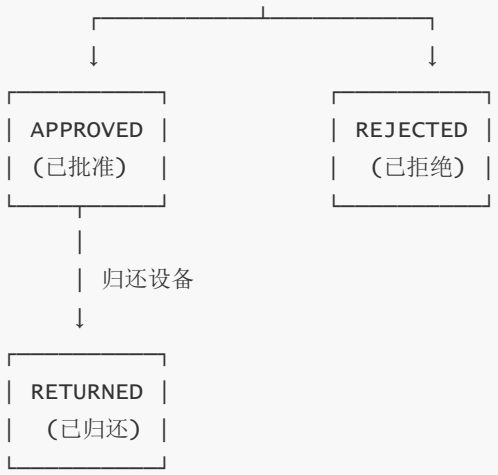
表5：borrow_records（借用记录表）

表结构：

```sql
CREATE TABLE borrow_records (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
    user_id BIGINT NOT NULL,            -- 借用人ID（外键）
    device_id BIGINT NOT NULL,          -- 设备ID（外键）
    device_instance_id BIGINT,          -- 设备实例ID（外键）★核心字段★
    quantity INTEGER NOT NULL,          -- 借用数量（兼容字段，实际为1）
    purpose VARCHAR(255) NOT NULL,      -- 借用目的
    expected_return_date DATE NOT NULL,  -- 预计归还日期
    actual_return_date DATE,            -- 实际归还日期
    status VARCHAR(20) NOT NULL DEFAULT 'PENDING',  -- 状态：
PENDING/APPROVED/REJECTED/RETURNED
    approve_user_id BIGINT,             -- 审批人ID（外键）
    approve_time DATETIME,              -- 审批时间
    approve_remark VARCHAR(255),        -- 审批备注
    return_remark VARCHAR(255),         -- 归还备注
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES users(id),
    FOREIGN KEY (device_id) REFERENCES devices(id),
    FOREIGN KEY (device_instance_id) REFERENCES device_instances(id),
    FOREIGN KEY (approve_user_id) REFERENCES users(id)
);
```

状态流转图：





示例数据:

```
id | user_id | device_id | device_instance_id | purpose    | status   |
approve_user_id
---|---------|-----------|--------------------|------------|----------|-----------
-----
16 | 2       | 1         | 1                  | 拍摄作业    | RETURNED | 2
17 | 2       | 6         | 4                  | 拍花        | RETURNED | 2
18 | 2       | 2         | 2                  | 测试        | REJECTED | 2
```

3.5 配置文件详解

application.properties（核心配置）

```
# 应用名称
spring.application.name=DeviceManage-backend

# 服务器配置
server.port=9011                    # 后端运行在9011端口
server.address=0.0.0.0              # 监听所有网络接口，允许外部访问

# 数据库配置
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
spring.datasource.url=jdbc:mysql://localhost:3306/device_manage?
useUnicode=true&characterEncoding=utf8&useSSL=false&serverTimezone=Asia/Shanghai
spring.datasource.username=root     # 数据库用户名
spring.datasource.password=123456   # 数据库密码

# MyBatis Plus 配置
mybatis-plus.configuration.map-underscore-to-camel-case=true  # 下划线转驼峰
mybatis-plus.mapper-locations=classpath*:/mapper/**/*.xml     # Mapper XML文件位置
mybatis-plus.type-aliases-package=chat.hbcit.devicemanagebackend.entity  # 实体类
包名

# JWT 配置
jwt.secret=DeviceManageBackendSecretKey2024ForJWTTokenGeneration  # JWT密钥
jwt.expiration=86400000  # Token过期时间（毫秒），86400000 = 24小时

# 日志配置
```

```
logging.level.root=INFO  # 根日志级别
logging.level.chat.hbcit.devicemanagebackend=DEBUG  # 项目日志级别
```

理解要点：

- `server.port`：后端运行的端口号，前端会访问这个端口
- `datasource.url`：数据库地址，`localhost:3306` 是本地MySQL
- `jwt.secret`：JWT签名密钥，必须保密，不能泄露
- `jwt.expiration`：Token有效期，24小时后需要重新登录

---

pom.xml（Maven依赖配置）

```
<!-- Maven是Java项目的依赖管理工具 -->
<!-- pom.xml定义了项目需要哪些库 -->

<dependencies>
    <!-- Spring Boot Web：提供Web功能 -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>

    <!-- Spring Security：提供安全认证功能 -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-security</artifactId>
    </dependency>

    <!-- MyBatis Plus：数据库操作工具 -->
    <dependency>
        <groupId>com.baomidou</groupId>
        <artifactId>mybatis-plus-spring-boot3-starter</artifactId>
        <version>3.5.9</version>
    </dependency>

    <!-- MySQL驱动：连接MySQL数据库 -->
    <dependency>
        <groupId>com.mysql</groupId>
        <artifactId>mysql-connector-j</artifactId>
    </dependency>

    <!-- JWT：生成和验证Token -->
    <dependency>
        <groupId>io.jsonwebtoken</groupId>
        <artifactId>jjwt-api</artifactId>
        <version>0.12.3</version>
    </dependency>

    <!-- Lombok：自动生成getter/setter等代码 -->
    <dependency>
        <groupId>org.projectlombok</groupId>
        <artifactId>lombok</artifactId>
    </dependency>
```

```
</dependencies>
```

理解要点:

- Maven会自动下载这些依赖库
- 不需要手动下载jar包
- 版本号统一管理，避免冲突

---

4. 前端技术详解

4.1 前端用到的技术

技术1：HTML（页面结构）

是什么?

- HTML定义页面的结构和内容
- 就像房子的框架

我们项目中的例子:

```
<!DOCTYPE html>
<html lang="zh-CN">
<head>
    <meta charset="UTF-8">
    <title>设备浏览 - 光影资产</title>
    <link rel="stylesheet" href="../css/common.css">  <!-- 引入样式 -->
</head>
<body>
    <!-- 页面标题 -->
    <div class="page-header">
        <h1 class="page-title">设备浏览</h1>
        <p class="page-description">浏览可用设备，申请借用</p>
    </div>

    <!-- 设备列表容器 -->
    <div id="deviceList"></div>

    <!-- 引入JavaScript -->
    <script src="../js/api.js"></script>
    <script src="../js/auth.js"></script>
</body>
</html>
```

技术2：CSS（页面样式）

是什么?

- CSS定义页面的外观和样式
- 就像房子的装修

我们项目中的特色：毛玻璃效果

```css
/* 卡片样式 */
.card {
    background-color: rgba(255, 255, 255, 0.85);  /* 半透明白色背景 */
    backdrop-filter: blur(20px);                  /* 模糊背景 */
    -webkit-backdrop-filter: blur(20px);          /* Safari浏览器兼容 */
    border-radius: 12px;                          /* 圆角 */
    box-shadow: 0 4px 6px rgba(0, 0, 0, 0.1);     /* 阴影 */
    padding: 24px;
}
```

效果说明：

- `rgba(255, 255, 255, 0.85)`：白色，85%不透明度（15%透明）
- `backdrop-filter: blur(20px)`：背景模糊20像素，产生毛玻璃效果
- 让界面看起来现代、高级

响应式设计：

```css
/* 电脑屏幕：3列布局 */
.device-grid {
    display: grid;
    grid-template-columns: repeat(3, 1fr);  /* 3列，每列等宽 */
    gap: 24px;
}

/* 手机屏幕：1列布局 */
@media (max-width: 768px) {
    .device-grid {
        grid-template-columns: 1fr;  /* 改为1列 */
    }
}
```

技术3：JavaScript（页面交互）

是什么?

- JavaScript让页面可以响应用户操作
- 可以发送请求、动态更新页面

核心功能1：发送HTTP请求

```javascript
// 文件：api.js

// 后端API地址
const API_BASE_URL = 'http://localhost:9011';

// 统一请求函数
async function request(url, options = {}) {
    const token = localStorage.getItem('token');  // 从本地存储获取Token

    // 设置请求头
    const headers = {
        'Content-Type': 'application/json',
```

```
        'Authorization': `Bearer ${token}`  // 携带Token
    };

    // 发送请求
    const response = await fetch(`${API_BASE_URL}${url}`, {
        ...options,
        headers: headers
    });

    // 处理响应
    const data = await response.json();
    return data;
}

// GET请求（查询数据）
async function get(url, params = {}) {
    const queryString = new URLSearchParams(params).toString();
    const finalUrl = queryString ? `${url}?${queryString}` : url;
    return request(finalUrl, { method: 'GET' });
}

// POST请求（创建数据）
async function post(url, data = {}) {
    return request(url, {
        method: 'POST',
        body: JSON.stringify(data)  // 把对象转换成JSON字符串
    });
}
```

使用示例:

```
// 查询设备列表
const response = await get('/devices', { page: 1, size: 10 });
console.log(response.data.list);  // 打印设备列表

// 提交借用申请
const response = await post('/borrow/apply', {
    deviceId: 1,
    purpose: '拍摄作业',
    expectedReturnDate: '2025-12-20'
});
```

核心功能2：动态生成HTML

```
// 文件：devices.html

// 渲染设备列表
function renderDevices(devices) {
    const container = document.getElementById('deviceList');

    // 创建设备网格容器
    const grid = document.createElement('div');
    grid.className = 'device-grid';
```

```
    // 遍历每个设备，创建卡片
    devices.forEach(device => {
        const card = document.createElement('div');
        card.className = 'device-card';

        // 设置卡片的HTML内容
        card.innerHTML = `
            <div class="device-info">
                <div class="device-category">${device.categoryName}</div>
                <h3 class="device-name">${device.name}</h3>
                <p class="device-model">型号：${device.model}</p>
                <p class="device-description">${device.description}</p>
                <button class="btn btn-primary" data-id="${device.id}">
                    申请借用
                </button>
            </div>
        `;

        // 添加点击事件
        const btn = card.querySelector('button');
        btn.addEventListener('click', () => {
            openBorrowModal(device.id, device.name);
        });

        grid.appendChild(card);
    });

    // 把网格添加到容器中
    container.innerHTML = '';
    container.appendChild(grid);
}
```

理解要点：

- `document.getElementById`：根据ID获取HTML元素
- `document.createElement`：创建新的HTML元素
- `innerHTML`：设置元素的HTML内容
- `addEventListener`：添加事件监听器（点击、输入等）

核心功能3：异步加载数据

```
// 加载设备列表（异步函数）
async function loadDevices() {
    const container = document.getElementById('deviceList');

    // 显示加载状态
    container.innerHTML = '<div class="loading">加载中...</div>';

    try {
        // 发送请求到后端
        const response = await API.deviceBrowse.list({
            page: 1,
```

```
            size: 12
        });

        // 检查响应
        if (response.code === 200) {
            // 成功：渲染设备列表
            renderDevices(response.data.list);
        } else {
            // 失败：显示错误信息
            container.innerHTML = `<div class="error">${response.message}</div>`;
        }
    } catch (error) {
        // 异常：显示错误信息
        container.innerHTML = `<div class="error">加载失败：${error.message}
</div>`;
    }
}

// 页面加载完成后，自动调用
document.addEventListener('DOMContentLoaded', function() {
    loadDevices();  // 加载设备列表
});
```

理解要点：

- `async/await`：异步编程语法，等待请求完成
- `try-catch`：捕获异常，防止程序崩溃
- `DOMContentLoaded`：页面加载完成后触发

4.2 前端文件详解

文件1：index.html（登录页面）

功能：用户输入用户名和密码，点击登录

核心代码：

```
<!-- 登录表单 -->
<form id="loginForm">
    <div class="form-group">
        <label>用户名</label>
        <input type="text" id="username" placeholder="请输入用户名" required>
    </div>
    <div class="form-group">
        <label>密码</label>
        <input type="password" id="password" placeholder="请输入密码" required>
    </div>
    <button type="submit" class="btn-primary">登录</button>
</form>

<script src="js/login.js"></script>
```

```
// 文件：login.js
```

```
// 监听表单提交事件
document.getElementById('loginForm').addEventListener('submit', async function(e)
{
    e.preventDefault();  // 阻止表单默认提交行为

    // 获取用户输入
    const username = document.getElementById('username').value.trim();
    const password = document.getElementById('password').value;

    // 验证输入
    if (!username || !password) {
        alert('请输入用户名和密码');
        return;
    }

    try {
        // 调用登录API
        const response = await API.auth.login(username, password);

        if (response.code === 200) {
            // 登录成功：保存Token和用户信息
            localStorage.setItem('token', response.data.token);
            localStorage.setItem('userInfo',
JSON.stringify(response.data.userInfo));

            // 根据角色跳转到不同页面
            if (response.data.userInfo.role === 'ADMIN' ||
                response.data.userInfo.role === 'ROOT') {
                window.location.href = '/admin/dashboard.html';  // 管理员 → 仪表盘
            } else {
                window.location.href = '/user/devices.html';     // 普通用户 → 设备
浏览
            }
        } else {
            alert(response.message || '登录失败');
        }
    } catch (error) {
        alert('登录失败：' + error.message);
    }
});
```

文件2：user/devices.html（设备浏览页面）

功能：显示所有可借用的设备，用户可以申请借用

页面流程：

```
1. 页面加载
   ↓
2. 检查用户是否登录（requireAuth）
   ↓
3. 创建导航栏（createLayout）
   ↓
4. 加载分类列表（loadCategories）
   ↓
5. 加载设备列表（loadDevices）
   ↓
6. 用户点击"申请借用" → 打开借用表单
   ↓
7. 用户填写信息并提交 → 调用API → 刷新列表
```

核心代码片段:

```javascript
// 加载设备列表
async function loadDevices() {
    const container = document.getElementById('deviceList');
    showLoading(container);  // 显示加载动画

    try {
        // 调用后端API
        const response = await API.deviceBrowse.list({
            page: currentPage,
            size: pageSize,
            name: searchKeyword,         // 搜索关键词
            categoryId: selectedCategoryId  // 选中的分类
        });

        if (response.code === 200) {
            renderDevices(response.data.list, response.data.total);
        }
    } catch (error) {
        container.innerHTML = showEmptyState('加载失败');
    }
}

// 渲染设备卡片
function renderDevices(devices, total) {
    const grid = document.createElement('div');
    grid.className = 'device-grid';

    devices.forEach(device => {
        const card = document.createElement('div');
        card.className = 'device-card';
        card.innerHTML = `
            <h3>${device.name}</h3>
            <p>型号：${device.model}</p>
            <button data-id="${device.id}">申请借用</button>
        `;

        // 绑定点击事件
        card.querySelector('button').addEventListener('click', () => {
            openBorrowModal(device.id, device.name);
```

```
        });

        grid.appendChild(card);
    });

    container.appendChild(grid);
}
```

文件3：admin/borrow-approval.html（借用审批页面）

功能：管理员查看待审批的申请，批准或拒绝

核心代码：

```
// 加载待审批申请
async function loadPendingRecords() {
    const response = await API.borrowAdmin.pending({
        page: currentPage,
        size: pageSize
    });

    if (response.code === 200) {
        renderRecords(response.data.list);
    }
}

// 渲染申请列表（表格形式）
function renderRecords(records) {
    const table = document.createElement('table');
    table.innerHTML = `
        <thead>
            <tr>
                <th>申请人</th>
                <th>设备名称</th>
                <th>借用目的</th>
                <th>预计归还</th>
                <th>操作</th>
            </tr>
        </thead>
        <tbody>
            ${records.map(record => `
                <tr>
                    <td>${record.realName}</td>
                    <td>${record.deviceName}</td>
                    <td>${record.purpose}</td>
                    <td>${record.expectedReturnDate}</td>
                    <td>
                        <button onclick="approve(${record.id}, true)">批准
</button>
                        <button onclick="approve(${record.id}, false)">拒绝
</button>
                    </td>
                </tr>
            `).join('')}
```

```
            </tbody>
        `;
        container.appendChild(table);
    }

    // 审批申请
    async function approve(recordId, isApprove) {
        const action = isApprove ? 'APPROVE' : 'REJECT';

        if (!confirm(`确认${isApprove ? '批准' : '拒绝'}该申请吗？`)) {
            return;
        }

        try {
            const response = await API.borrowAdmin.approve(recordId, {
                action: action,
                remark: ''
            });

            if (response.code === 200) {
                alert('操作成功');
                loadPendingRecords();  // 刷新列表
            }
        } catch (error) {
            alert('操作失败：' + error.message);
        }
    }
```

4.3 前端和后端的数据交互

示例1：查询设备列表

前端代码：

```
// 发送请求
const response = await fetch('http://localhost:9011/devices?page=1&size=10', {
    method: 'GET',
    headers: {
        'Authorization': 'Bearer ' + token
    }
});

// 解析响应
const data = await response.json();
console.log(data);
```

后端接收到的请求：

```
GET /devices?page=1&size=10
Headers:
  Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

后端Controller处理：

```java
@GetMapping
public Result<PageResult<DeviceVO>> getDeviceList(
        @RequestParam(defaultValue = "1") Integer page,
        @RequestParam(defaultValue = "10") Integer size
) {
    // page = 1, size = 10（从请求参数中获取）
    PageResult<DeviceVO> result = deviceService.getDeviceList(page, size);
    return Result.success("获取成功", result);
}
```

后端返回的响应:

```json
{
    "code": 200,
    "message": "获取成功",
    "data": {
        "total": 15,
        "page": 1,
        "size": 10,
        "list": [
            {
                "id": 1,
                "name": "佳能 EOS R5",
                "model": "EOS R5",
                "categoryName": "相机",
                "overallStatus": "NORMAL"
            },
            ...
        ]
    }
}
```

前端接收并显示:

```javascript
if (response.code === 200) {
    const devices = response.data.list;

    // 动态创建HTML
    devices.forEach(device => {
        const card = document.createElement('div');
        card.innerHTML = `
            <h3>${device.name}</h3>
            <p>${device.model}</p>
        `;
        container.appendChild(card);
    });
}
```

示例2：提交借用申请

前端代码：

```
// 用户点击"提交申请"按钮
async function submitBorrow() {
    const deviceId = document.getElementById('borrowDeviceId').value;
    const purpose = document.getElementById('borrowPurpose').value;
    const expectedReturnDate =
document.getElementById('expectedReturnDate').value;

    // 调用API
    const response = await API.borrow.apply({
        deviceId: parseInt(deviceId),
        purpose: purpose,
        expectedReturnDate: expectedReturnDate
    });

    if (response.code === 200) {
        alert('申请提交成功');
        closeBorrowModal();  // 关闭弹窗
        loadDevices();       // 刷新设备列表
    } else {
        alert(response.message);
    }
}
```

后端接收并处理：

```
@PostMapping("/borrow/apply")
public Result<BorrowRecord> applyBorrow(@RequestBody BorrowApplyRequest request)
{
    // 1. 从Token中获取当前用户ID
    Authentication authentication =
SecurityContextHolder.getContext().getAuthentication();
    Long userId = Long.valueOf(authentication.getName());

    // 2. 创建借用记录对象
    BorrowRecord record = new BorrowRecord();
    record.setDeviceId(request.getDeviceId());
    record.setPurpose(request.getPurpose());
    record.setExpectedReturnDate(request.getExpectedReturnDate());

    // 3. 调用Service处理业务逻辑
    BorrowRecord createdRecord = borrowService.applyBorrow(userId, record);

    // 4. 返回结果
    return Result.success("申请提交成功", createdRecord);
}
```

Service层的处理：

```
@Transactional  // 开启事务
public BorrowRecord applyBorrow(Long userId, BorrowRecord record) {
```

```
    // 1. 验证设备是否存在
    Device device = deviceMapper.selectById(record.getDeviceId());
    if (device == null) {
        throw new BusinessException("设备不存在");
    }

    // 2. 查找可用的设备实例
    List<DeviceInstance> instances = deviceInstanceMapper
        .selectWithBorrowStatus(record.getDeviceId());

    DeviceInstance availableInstance = null;
    for (DeviceInstance instance : instances) {
        // 检查状态是否为正常
        if ("NORMAL".equals(instance.getStatus())) {
            // 检查是否已被借用
            Long borrowCount = borrowRecordMapper.selectCount(
                new LambdaQueryWrapper<BorrowRecord>()
                    .eq(BorrowRecord::getDeviceInstanceId, instance.getId())
                    .in(BorrowRecord::getStatus, "PENDING", "APPROVED")
            );

            if (borrowCount == 0) {
                availableInstance = instance;  // 找到可用实例
                break;
            }
        }
    }

    if (availableInstance == null) {
        throw new BusinessException("该设备暂无可用实例");
    }

    // 3. 创建借用记录
    record.setUserId(userId);
    record.setDeviceInstanceId(availableInstance.getId());
    record.setStatus("PENDING");  // 状态：待审批
    borrowRecordMapper.insert(record);

    return record;
}
```

5. 核心业务流程

5.1 用户登录流程（完整版）

```
┌─────────────────────────────────────────────────────────────┐
│  第1步：用户在浏览器输入用户名和密码                          │
├─────────────────────────────────────────────────────────────┤
│  前端：index.html                                           │
│  用户输入：username = "admin", password = "123456"          │
│  点击"登录"按钮                                             │
└─────────────────────────────────────────────────────────────┘
                              ↓
```

```
┌──────────────────────────────────────────────────────────────┐
│  第2步：前端发送登录请求到后端                                  │
├──────────────────────────────────────────────────────────────┤
│  前端：login.js                                               │
│  const response = await API.auth.login(username, password);  │
│                                                              │
│  实际发送的HTTP请求：                                          │
│  POST http://localhost:9011/auth/login                       │
│  Content-Type: application/json                              │
│  Body: {"username": "admin", "password": "123456"}           │
└──────────────────────────────────────────────────────────────┘
                              ↓
┌──────────────────────────────────────────────────────────────┐
│  第3步：后端Controller接收请求                                 │
├──────────────────────────────────────────────────────────────┤
│  后端：AuthController.java                                    │
│  @PostMapping("/login")                                      │
│  public Result<LoginResponse> login(@RequestBody LoginRequest request) {|
│      LoginResponse response = authService.login(request);    │
│      return Result.success("登录成功", response);             │
│  }                                                           │
└──────────────────────────────────────────────────────────────┘
                              ↓
┌──────────────────────────────────────────────────────────────┐
│  第4步：Service验证用户名和密码                                │
├──────────────────────────────────────────────────────────────┤
│  后端：AuthServiceImpl.java                                   │
│                                                              │
│  1. 根据用户名查询数据库                                       │
│     User user = userMapper.selectOne(                        │
│         new LambdaQueryWrapper<User>()                       │
│             .eq(User::getUsername, request.getUsername())    │
│     );                                                       │
│                                                              │
│  2. 检查用户是否存在                                           │
│     if (user == null) {                                      │
│         throw new BusinessException("用户名或密码错误");        │
│     }                                                        │
│                                                              │
│  3. 验证密码                                                  │
│     boolean isMatch = passwordEncoder.matches(               │
│         request.getPassword(),  // 用户输入的密码              │
│         user.getPassword()      // 数据库中的加密密码          │
│     );                                                       │
│     if (!isMatch) {                                          │
│         throw new BusinessException("用户名或密码错误");        │
│     }                                                        │
│                                                              │
│  4. 检查用户状态                                              │
│     if (!"ACTIVE".equals(user.getStatus())) {                │
│         throw new BusinessException("用户已被禁用");           │
│     }                                                        │
└──────────────────────────────────────────────────────────────┘
                              ↓
┌──────────────────────────────────────────────────────────────┐
│  第5步：生成JWT Token                                         │
```

```
┌──────────────────────────────────────────────────────────────┐
│  后端：JwtUtil.java                                          │
│                                                              │
│  String token = jwtUtil.generateToken(                       │
│      user.getId(),      // 用户ID: 1                         │
│      user.getUsername(), // 用户名: "admin"                  │
│      user.getRole()     // 角色: "ADMIN"                     │
│  );                                                          │
│                                                              │
│  生成的Token（示例）：                                       │
│
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySWQiOjEsInVzZXJuYW1lIjoiYWRtaW4iLCJy
b2xlIjoiQURNSU4ifQ.xxxxx│
│                                                              │
│  Token内容（解码后）：                                       │
│  {                                                           │
│      "userId": 1,                                            │
│      "username": "admin",                                    │
│      "role": "ADMIN",                                        │
│      "iat": 1702800000,  // 签发时间                         │
│      "exp": 1702886400   // 过期时间（24小时后）             │
│  }                                                           │
└──────────────────────────────────────────────────────────────┘
                              ↓
┌──────────────────────────────────────────────────────────────┐
│  第6步：返回Token和用户信息给前端                            │
├──────────────────────────────────────────────────────────────┤
│  后端返回的JSON：                                            │
│  {                                                           │
│      "code": 200,                                            │
│      "message": "登录成功",                                  │
│      "data": {                                               │
│          "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...", │
│          "userInfo": {                                       │
│              "id": 1,                                        │
│              "username": "admin",                            │
│              "realName": "管理员",                           │
│              "role": "ADMIN"                                 │
│          }                                                   │
│      }                                                       │
│  }                                                           │
└──────────────────────────────────────────────────────────────┘
                              ↓
┌──────────────────────────────────────────────────────────────┐
│  第7步：前端保存Token和用户信息                              │
├──────────────────────────────────────────────────────────────┤
│  前端：login.js                                              │
│                                                              │
│  localStorage.setItem('token', response.data.token);         │
│  localStorage.setItem('userInfo', JSON.stringify(response.data.userInfo));│
│                                                              │
│  // Token和用户信息保存在浏览器的localStorage中              │
│  // 即使关闭浏览器，下次打开还在（除非清除缓存）             │
└──────────────────────────────────────────────────────────────┘
                              ↓
┌──────────────────────────────────────────────────────────────┐
```

```
|  第8步：跳转到对应页面                                                    |
├──────────────────────────────────────────────────────────────────────┤
|  if (userInfo.role === 'ADMIN' || userInfo.role === 'ROOT') {  |
|      window.location.href = '/admin/dashboard.html';  // 管理员|
|  } else {                                                             |
|      window.location.href = '/user/devices.html';     // 用户  |
|  }                                                                    |
└──────────────────────────────────────────────────────────────────────┘
```

5.2 借用申请流程（完整版）

```
┌──────────────────────────────────────────────────────────────────────┐
|  第1步：用户浏览设备列表                                              |
├──────────────────────────────────────────────────────────────────────┤
|  页面：user/devices.html                                              |
|  用户看到所有可借用的设备（卡片式展示）                               |
|  每个设备卡片显示：名称、型号、分类、状态                             |
└──────────────────────────────────────────────────────────────────────┘
                                ↓
┌──────────────────────────────────────────────────────────────────────┐
|  第2步：用户点击"申请借用"按钮                                        |
├──────────────────────────────────────────────────────────────────────┤
|  前端弹出借用申请表单（模态框）                                       |
|  表单包含：                                                           |
|    - 设备名称（自动填充，只读）                                       |
|    - 借用目的（用户填写）                                             |
|    - 预计归还日期（用户选择）                                         |
└──────────────────────────────────────────────────────────────────────┘
                                ↓
┌──────────────────────────────────────────────────────────────────────┐
|  第3步：用户填写信息并提交                                            |
├──────────────────────────────────────────────────────────────────────┤
|  用户输入：                                                           |
|    - 借用目的："拍摄期末作业"                                         |
|    - 预计归还日期："2025-12-25"                                       |
|  点击"提交申请"按钮                                                   |
└──────────────────────────────────────────────────────────────────────┘
                                ↓
┌──────────────────────────────────────────────────────────────────────┐
|  第4步：前端发送请求到后端                                            |
├──────────────────────────────────────────────────────────────────────┤
|  POST http://localhost:9011/borrow/apply                              |
|  Headers:                                                             |
|    Authorization: Bearer eyJhbGci...                                  |
|  Body:                                                                |
|  {                                                                    |
|      "deviceId": 1,                                                   |
|      "purpose": "拍摄期末作业",                                       |
|      "expectedReturnDate": "2025-12-25"                               |
|  }                                                                    |
└──────────────────────────────────────────────────────────────────────┘
                                ↓
┌──────────────────────────────────────────────────────────────────────┐
|  第5步：后端验证Token并获取用户ID                                     |
```

```
┌──────────────────────────────────────────────────────────────┐
│  后端：JwtAuthenticationFilter.java（过滤器）                  │
│                                                              │
│  1. 从请求头中提取Token                                       │
│     String token = request.getHeader("Authorization")        │
│                          .substring(7);  // 去掉"Bearer "    │
│                                                              │
│  2. 验证Token是否有效                                         │
│     if (jwtUtil.validateToken(token)) {                      │
│         Long userId = jwtUtil.getUserIdFromToken(token);     │
│         String role = jwtUtil.getRoleFromToken(token);       │
│         // 设置到Spring Security上下文                        │
│         SecurityContextHolder.getContext()                   │
│             .setAuthentication(authentication);              │
│     }                                                        │
│                                                              │
│  3. 如果Token无效，返回401错误                                │
└──────────────────────────────────────────────────────────────┘
                              ↓
┌──────────────────────────────────────────────────────────────┐
│  第6步：Controller接收请求并调用Service                       │
├──────────────────────────────────────────────────────────────┤
│  后端：BorrowController.java                                 │
│                                                              │
│  @PostMapping("/borrow/apply")                               │
│  public Result<BorrowRecord> applyBorrow(                    │
│          @RequestBody BorrowApplyRequest request) {          │
│                                                              │
│      // 从Security上下文获取当前用户ID                        │
│      Authentication auth = SecurityContextHolder             │
│          .getContext().getAuthentication();                  │
│      Long userId = Long.valueOf(auth.getName());             │
│      // userId = 10（赵杰的用户ID）                           │
│                                                              │
│      // 调用Service处理                                       │
│      BorrowRecord record = borrowService.applyBorrow(userId, request);│
│                                                              │
│      return Result.success("申请提交成功", record);           │
│  }                                                           │
└──────────────────────────────────────────────────────────────┘
                              ↓
┌──────────────────────────────────────────────────────────────┐
│  第7步：Service处理业务逻辑（核心）                            │
├──────────────────────────────────────────────────────────────┤
│  后端：BorrowServiceImpl.java                                │
│                                                              │
│  @Transactional  // 开启事务，保证数据一致性                   │
│  public BorrowRecord applyBorrow(Long userId, BorrowRecord record) {│
│                                                              │
│      // 步骤1：验证设备是否存在                               │
│      Device device = deviceMapper.selectById(record.getDeviceId());│
│      if (device == null) {                                   │
│          throw new BusinessException("设备不存在");           │
│      }                                                       │
│                                                              │
│      // 步骤2：查询该设备的所有实例                           │
```

```
|      List<DeviceInstance> instances = deviceInstanceMapper     |
|          .selectWithBorrowStatus(record.getDeviceId());        |
|      // 假设查询到3个实例：001、002、003                          |
|                                                                 |
|      // 步骤3：筛选可用的实例                                     |
|      DeviceInstance availableInstance = null;                   |
|      for (DeviceInstance instance : instances) {                |
|          // 3.1 检查实例状态是否为正常                            |
|          if (!"NORMAL".equals(instance.getStatus())) {          |
|              continue;  // 跳过维修中或已报废的实例               |
|          }                                                      |
|                                                                 |
|          // 3.2 检查实例是否已被借用                              |
|          Long borrowCount = borrowRecordMapper.selectCount(     |
|              new LambdaQueryWrapper<BorrowRecord>()             |
|                  .eq(BorrowRecord::getDeviceInstanceId, instance.getId())|
|                  .in(BorrowRecord::getStatus, "PENDING", "APPROVED")|
|          );                                                     |
|                                                                 |
|          if (borrowCount == 0) {                                |
|              availableInstance = instance;  // 找到可用实例!     |
|              break;  // 找到第一个就停止                          |
|          }                                                      |
|      }                                                          |
|                                                                 |
|      // 步骤4：如果没有可用实例，抛出异常                          |
|      if (availableInstance == null) {                           |
|          throw new BusinessException("该设备暂无可用实例");      |
|      }                                                          |
|                                                                 |
|      // 步骤5：创建借用记录                                       |
|      record.setUserId(userId);  // 设置借用人ID                  |
|      record.setDeviceInstanceId(availableInstance.getId());     |
|      record.setStatus("PENDING");  // 状态：待审批               |
|      record.setQuantity(1);  // 数量固定为1                      |
|      borrowRecordMapper.insert(record);  // 插入数据库           |
|                                                                 |
|      return record;                                             |
|  }                                                              |
└─────────────────────────────────────────────────────────────────┘
                              ↓
┌─────────────────────────────────────────────────────────────────┐
|  第8步：Mapper执行SQL插入数据库                                   |
├─────────────────────────────────────────────────────────────────┤
|  INSERT INTO borrow_records (                                   |
|      user_id, device_id, device_instance_id,                     |
|      purpose, expected_return_date, status, quantity            |
|  ) VALUES (                                                     |
|      10,  -- 赵杰的用户ID                                        |
|      1,   -- 佳能EOS R5的设备ID                                  |
|      1,   -- 佳能EOS R5-001的实例ID                              |
|      '拍摄期末作业',                                              |
|      '2025-12-25',                                              |
|      'PENDING',                                                 |
|      1                                                          |
|  );                                                             |
```

```
┌──────────────────────────────────────────────────────────────┐
│                                                              │
│  数据库自动生成：                                            │
│    - id = 26（自动增长）                                     │
│    - created_at = 当前时间                                   │
│    - updated_at = 当前时间                                   │
└──────────────────────────────────────────────────────────────┘
                              ↓
┌──────────────────────────────────────────────────────────────┐
│  第9步：返回结果给前端                                       │
├──────────────────────────────────────────────────────────────┤
│  后端返回的JSON：                                            │
│  {                                                           │
│      "code": 200,                                            │
│      "message": "申请提交成功",                              │
│      "data": {                                               │
│          "id": 26,                                           │
│          "userId": 10,                                       │
│          "deviceId": 1,                                      │
│          "deviceInstanceId": 1,                              │
│          "purpose": "拍摄期末作业",                          │
│          "expectedReturnDate": "2025-12-25",                 │
│          "status": "PENDING",                                │
│          "createdAt": "2025-12-18 20:30:00"                  │
│      }                                                       │
│  }                                                           │
└──────────────────────────────────────────────────────────────┘
                              ↓
┌──────────────────────────────────────────────────────────────┐
│  第10步：前端显示成功提示并刷新页面                          │
├──────────────────────────────────────────────────────────────┤
│  前端：devices.html                                          │
│                                                              │
│  if (response.code === 200) {                                │
│      alert('申请提交成功');  // 弹出提示                     │
│      closeBorrowModal();     // 关闭申请表单                 │
│      loadDevices();          // 重新加载设备列表             │
│  }                                                           │
└──────────────────────────────────────────────────────────────┘
```

5.3 借用审批流程（完整版）

```
┌──────────────────────────────────────────────────────────────┐
│  第1步：管理员打开借用审批页面                               │
├──────────────────────────────────────────────────────────────┤
│  页面：admin/borrow-approval.html                            │
│  自动加载待审批申请列表                                      │
└──────────────────────────────────────────────────────────────┘
                              ↓
┌──────────────────────────────────────────────────────────────┐
│  第2步：前端请求待审批申请列表                               │
├──────────────────────────────────────────────────────────────┤
│  GET http://localhost:9011/admin/borrow/pending?page=1&size=10 │
│  Headers: Authorization: Bearer eyJhbGci...                  │
└──────────────────────────────────────────────────────────────┘
```

```
                                ↓
┌──────────────────────────────────────────────────────────────────────┐
│  第3步：后端查询待审批申请                                            │
├──────────────────────────────────────────────────────────────────────┤
│  后端：AdminBorrowController.java → BorrowServiceImpl.java           │
│                                                                      │
│  执行的SQL（简化版）：                                                │
│  SELECT br.*, u.username, u.real_name,                               │
│         d.name as device_name, d.model as device_model,              │
│         di.instance_name as device_instance_name                     │
│  FROM borrow_records br                                              │
│  LEFT JOIN users u ON br.user_id = u.id                              │
│  LEFT JOIN devices d ON br.device_id = d.id                          │
│  LEFT JOIN device_instances di ON br.device_instance_id = di.id      │
│  WHERE br.status = 'PENDING'  -- 只查询待审批的                       │
│  ORDER BY br.created_at DESC                                         │
│  LIMIT 0, 10;                                                        │
│                                                                      │
│  查询结果：                                                           │
│  id=26, user_id=10, username="赵杰", device_name="佳能EOS R5",       │
│  device_instance_name="佳能EOS R5-001", purpose="拍摄期末作业",       │
│  expected_return_date="2025-12-25", status="PENDING"                 │
└──────────────────────────────────────────────────────────────────────┘
                                ↓
┌──────────────────────────────────────────────────────────────────────┐
│  第4步：前端显示申请列表（表格形式）                                   │
├──────────────────────────────────────────────────────────────────────┤
│  ┌────┬──────┬──────────┬──────────┬──────┬──────┐                   │
│  |申请人|设备名称|借用目的    |预计归还   |状态    |操作    |          │
│  ├────┼──────┼──────────┼──────────┼──────┼──────┤                   │
│  |赵杰  |佳能EOS R5|拍摄期末作业|2025-12-25|待审批  |批准/拒绝|        │
│  └────┴──────┴──────────┴──────────┴──────┴──────┘                   │
└──────────────────────────────────────────────────────────────────────┘
                                ↓
┌──────────────────────────────────────────────────────────────────────┐
│  第5步：管理员点击"批准"按钮                                          │
├──────────────────────────────────────────────────────────────────────┤
│  前端弹出确认对话框："确认批准该用户借用设备？"                        │
│  管理员点击"确定"                                                     │
└──────────────────────────────────────────────────────────────────────┘
                                ↓
┌──────────────────────────────────────────────────────────────────────┐
│  第6步：前端发送审批请求                                              │
├──────────────────────────────────────────────────────────────────────┤
│  PUT http://localhost:9011/admin/borrow/26/approve                   │
│  Headers: Authorization: Bearer eyJhbGci...                          │
│  Body:                                                               │
│  {                                                                   │
│      "action": "APPROVE",                                            │
│      "remark": ""                                                    │
│  }                                                                   │
└──────────────────────────────────────────────────────────────────────┘
                                ↓
┌──────────────────────────────────────────────────────────────────────┐
│  第7步：后端处理审批逻辑                                              │
├──────────────────────────────────────────────────────────────────────┤
```

```
┌──────────────────────────────────────────────────────────────┐
│  后端：BorrowServiceImpl.java                                 │
│                                                              │
│  @Transactional  // 开启事务                                  │
│  public void approveBorrow(Long recordId, Long approveUserId,  │
│                            String action, String remark) {     │
│                                                              │
│      // 步骤1：查询借用记录                                    │
│      BorrowRecord record = borrowRecordMapper.selectById(recordId);│
│      // record.id = 26, status = "PENDING"                    │
│                                                              │
│      // 步骤2：验证状态是否为待审批                             │
│      if (!"PENDING".equals(record.getStatus())) {             │
│          throw new BusinessException("该申请已处理");           │
│      }                                                       │
│                                                              │
│      // 步骤3：如果是批准操作                                   │
│      if ("APPROVE".equals(action)) {                         │
│          // 3.1 检查设备实例是否可用                           │
│          DeviceInstance instance = deviceInstanceMapper        │
│              .selectById(record.getDeviceInstanceId());       │
│          if (!"NORMAL".equals(instance.getStatus())) {        │
│              throw new BusinessException("设备实例不可借用");   │
│          }                                                   │
│                                                              │
│          // 3.2 检查是否已被其他申请借用                        │
│          Long borrowCount = borrowRecordMapper.selectCount(    │
│              new LambdaQueryWrapper<BorrowRecord>()            │
│                  .eq(BorrowRecord::getDeviceInstanceId,        │
│                      record.getDeviceInstanceId())             │
│                  .in(BorrowRecord::getStatus, "PENDING", "APPROVED")│
│                  .ne(BorrowRecord::getId, recordId)            │
│          );                                                  │
│          if (borrowCount > 0) {                              │
│              throw new BusinessException("设备实例已被借用");   │
│          }                                                   │
│                                                              │
│          // 3.3 更新借用记录状态                               │
│          record.setStatus("APPROVED");  // 状态改为"已批准"     │
│          record.setApproveUserId(approveUserId);  // 记录审批人│
│          record.setApproveTime(LocalDateTime.now());  // 审批时间│
│          record.setApproveRemark(remark);  // 审批备注          │
│          borrowRecordMapper.updateById(record);  // 更新数据库  │
│      }                                                       │
│  }                                                           │
└──────────────────────────────────────────────────────────────┘
                              ↓
┌──────────────────────────────────────────────────────────────┐
│  第8步：数据库更新                                            │
├──────────────────────────────────────────────────────────────┤
│  UPDATE borrow_records                                       │
│  SET status = 'APPROVED',                                    │
│      approve_user_id = 2,  -- 管理员ID                        │
│      approve_time = '2025-12-18 20:35:00',                   │
│      approve_remark = '',                                    │
│      updated_at = '2025-12-18 20:35:00'                      │
│  WHERE id = 26;                                              │
```

```
|                                                                    |
|  更新后的记录：                                                    |
|  id=26, status="APPROVED", approve_user_id=2,                      |
|  approve_time="2025-12-18 20:35:00"                                |
└────────────────────────────────────────────────────────────────────┘
                                ↓
┌────────────────────────────────────────────────────────────────────┐
|  第9步：返回成功响应                                               |
├────────────────────────────────────────────────────────────────────┤
|  {                                                                 |
|      "code": 200,                                                  |
|      "message": "审批成功",                                        |
|      "data": null                                                  |
|  }                                                                 |
└────────────────────────────────────────────────────────────────────┘
                                ↓
┌────────────────────────────────────────────────────────────────────┐
|  第10步：前端刷新列表                                              |
├────────────────────────────────────────────────────────────────────┤
|  前端：borrow-approval.html                                        |
|                                                                    |
|  if (response.code === 200) {                                      |
|      loadPendingRecords();  // 重新加载待审批列表                  |
|  }                                                                 |
|                                                                    |
|  此时，id=26的记录已经不在待审批列表中了                           |
|  （因为状态变成了APPROVED）                                       |
└────────────────────────────────────────────────────────────────────┘
```

5.4 设备状态计算逻辑（重要）

问题：一个设备有多个实例，如何显示设备的整体状态？

解决方案：根据所有实例的状态，计算设备的整体状态

计算规则：

```java
private String calculateDeviceStatus(List<DeviceInstanceVO> instances) {
    if (instances == null || instances.isEmpty()) {
        return "NORMAL";  // 没有实例，默认正常
    }

    boolean hasBorrowed = false;      // 是否有实例被借用
    boolean hasMaintenance = false;  // 是否有实例在维修
    boolean allScrapped = true;       // 是否所有实例都报废

    // 遍历所有实例
    for (DeviceInstanceVO inst : instances) {
        if (inst.getBorrowed()) {
            hasBorrowed = true;  // 有实例被借用
        }
        if ("MAINTENANCE".equals(inst.getStatus())) {
            hasMaintenance = true;  // 有实例在维修
        }
```

```
            if (!"SCRAPPED".equals(inst.getStatus())) {
                allScrapped = false;  // 不是所有实例都报废
            }
        }

        // 优先级：已报废 > 借用中 > 维修中 > 正常
        if (allScrapped) {
            return "SCRAPPED";  // 所有实例都报废 → 已报废
        }
        if (hasBorrowed) {
            return "BORROWED";  // 有实例被借用 → 借用中
        }
        if (hasMaintenance) {
            return "MAINTENANCE";  // 有实例在维修 → 维修中
        }
        return "NORMAL";  // 其他情况 → 正常
    }
```

举例说明：

```
设备：佳能 EOS R5
  ├─ 实例1：正常
  ├─ 实例2：借用中  ← 有实例被借用
  └─ 实例3：正常
→ 设备整体状态：借用中

设备：索尼 A7M4
  ├─ 实例1：正常
  ├─ 实例2：维修中  ← 有实例在维修
  └─ 实例3：正常
→ 设备整体状态：维修中

设备：尼康 Z9
  ├─ 实例1：已报废
  ├─ 实例2：已报废  ← 所有实例都报废
  └─ 实例3：已报废
→ 设备整体状态：已报废
```

---

6. 如何运行项目

6.1 环境准备

第1步：安装JDK 17

- 下载地址：https://www.oracle.com/java/technologies/downloads/
- 安装后，在命令行输入 `java -version` 检查是否安装成功

第2步：安装Maven

- 下载地址：https://maven.apache.org/download.cgi
- 配置环境变量
- 在命令行输入 `mvn -version` 检查是否安装成功

第3步：安装MySQL 8.0

- 下载地址：https://dev.mysql.com/downloads/mysql/
- 安装后，创建数据库：

```
CREATE DATABASE device_manage CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
```

第4步：导入数据库

- 打开 `docs/device_manage.sql` 文件
- 在MySQL中执行这个SQL脚本
- 会自动创建表和插入测试数据

6.2 启动后端

方法1：使用IDE（推荐）

步骤：

1. 用IntelliJ IDEA打开 `DeviceManage-backend` 文件夹
2. 等待Maven自动下载依赖（第一次会比较慢）
3. 打开 `src/main/resources/application.properties`
4. 修改数据库配置（用户名、密码）
5. 找到 `DeviceManageBackendApplication.java`
6. 右键 → Run 'DeviceManageBackendApplication'
7. 看到控制台输出 "Started DeviceManageBackendApplication" 表示启动成功
8. 后端运行在 `http://localhost:9011`

方法2：使用命令行

```
# 进入后端目录
cd DeviceManage-backend

# 编译项目
mvn clean package

# 运行项目
java -jar target/DeviceManage-backend-0.0.1-SNAPSHOT.jar
```

6.3 启动前端

方法1：使用Live Server（推荐）

步骤：

1. 安装VS Code
2. 安装Live Server插件

3. 用VS Code打开 `DeviceManage-frontend` 文件夹
4. 右键 `index.html` → Open with Live Server
5. 浏览器自动打开 `http://localhost:9012/index.html`

方法2：直接打开HTML文件

步骤：

1. 找到 `DeviceManage-frontend/index.html`
2. 双击打开（用浏览器打开）
3. 注意：这种方式可能有跨域问题

6.4 测试账号

超级管理员：

- 权限：最高权限，可以管理所有用户和设备

管理员：

- 用户名：`admin`
- 密码：`123456`
- 权限：可以管理设备、审批借用、管理普通用户

普通用户：

- 用户名：`user1`
- 密码：`user1`
- 权限：只能浏览设备和申请借用

6.5 完整测试流程

测试场景：完整的借用流程

步骤1：用户登录

1. 打开 `https://gyzc.hbcit.chat`
2. 输入用户名 `user1`，密码 `user1`
3. 点击"登录"
4. 自动跳转到设备浏览页面

步骤2：浏览设备

1. 看到所有可借用的设备（卡片式展示）
2. 可以按分类筛选，比如选择"相机"
3. 可以搜索，比如输入"佳能"

步骤3：申请借用

1. 找到"佳能 EOS R5"，点击"申请借用"

2. 弹出申请表单
3. 填写借用目的："拍摄期末作业"
4. 选择预计归还日期："2025-12-25"
5. 点击"提交申请"
6. 看到提示"申请提交成功"

步骤4：查看申请记录

1. 点击导航栏的"我的借用"
2. 看到刚才提交的申请，状态为"待审批"

步骤5：管理员审批

1. 退出登录，重新登录为管理员（用户名 `admin`，密码 `123456`）
2. 自动跳转到管理仪表盘
3. 看到"待审批申请"数量增加了1
4. 点击"借用审批"菜单
5. 在"待审批"列表中看到赵杰的申请
6. 点击"批准"按钮
7. 确认批准
8. 看到提示"审批成功"，申请从列表中消失

步骤6：用户查看结果

1. 切换回用户账号（或刷新页面）
2. 进入"我的借用"页面
3. 看到申请状态变成了"已批准"

步骤7：管理员确认归还

1. 用户使用完设备后，归还给管理员
2. 管理员在"借用审批"页面，切换到"全部记录"标签
3. 找到状态为"已批准"的记录
4. 点击"确认归还"按钮
5. 确认设备完好
6. 状态变成"已归还"

---

7. 常见问题解答

Q1：什么是前后端分离？为什么要分离？

答：

传统方式（不分离）：

```
用户访问 → 后端生成HTML页面 → 返回完整页面给用户
```

例如：JSP、Thymeleaf等模板引擎

前后端分离：

```
用户访问 → 前端HTML页面（静态） → JavaScript请求后端API → 后端返回JSON数据 → 前端渲染
```

为什么要分离？

1. **职责清晰**：前端负责展示，后端负责逻辑
2. **独立开发**：前端和后端可以并行开发，互不影响
3. **独立部署**：前端可以部署到CDN，后端可以部署到服务器
4. **易于维护**：修改界面不需要动后端，修改逻辑不需要动前端
5. **可扩展**：后端API可以给网页、手机App、小程序共用

Q2：什么是RESTful API?

答：

RESTful是一种API设计风格，核心思想是：

- 用URL表示资源（名词）
- 用HTTP方法表示操作（动词）

示例：

```
资源：用户（users）

GET    /users       → 获取所有用户（查询）
GET    /users/1     → 获取ID=1的用户（查询单个）
POST   /users       → 创建新用户（新增）
PUT    /users/1     → 更新ID=1的用户（修改）
DELETE /users/1     → 删除ID=1的用户（删除）
```

为什么这样设计？

- 语义清晰：一看URL就知道操作什么资源
- 统一规范：所有资源都用同样的方式操作
- 易于理解：符合HTTP协议的设计理念

Q3：什么是依赖注入（@Autowired）？

答：

传统方式（自己创建对象）：

```
public class UserServiceImpl {
    private UserMapper userMapper = new UserMapperImpl();  // 自己new
}
```

问题：

- 代码耦合度高

- 难以测试
- 难以替换实现

Spring方式（依赖注入）：

```
public class UserServiceImpl {
    @Autowired  // Spring自动注入
    private UserMapper userMapper;
}
```

好处：

- 不需要关心对象如何创建
- Spring容器统一管理
- 易于测试和替换

工作原理：

1. Spring启动时，扫描所有带 `@Component`、`@Service`、`@Repository` 等注解的类
2. 创建这些类的对象（Bean），放入Spring容器
3. 当遇到 `@Autowired` 时，从容器中找到对应的Bean注入进去

Q4：什么是事务（@Transactional）？

答：

事务：一组数据库操作，要么全部成功，要么全部失败（回滚）

为什么需要事务？

场景：批准借用申请

```
// 没有事务的情况（危险）
public void approveBorrow(Long recordId) {
    // 操作1：更新借用记录状态
    borrowRecordMapper.updateById(record);  // 成功

    // 操作2：减少设备库存
    deviceMapper.updateStock(deviceId);  // 失败（比如网络中断）

    // 结果：借用记录更新了，但库存没减少 → 数据不一致！
}
```

有事务的情况（安全）：

```
@Transactional  // 开启事务
public void approveBorrow(Long recordId) {
    // 操作1：更新借用记录状态
    borrowRecordMapper.updateById(record);

    // 操作2：减少设备库存
    deviceMapper.updateStock(deviceId);  // 如果这里失败

    // Spring会自动回滚操作1，保证数据一致性
}
```

事务的特性（ACID）：

- **A**tomicity（原子性）：要么全做，要么全不做
- **C**onsistency（一致性）：数据从一个一致状态到另一个一致状态
- **I**solation（隔离性）：多个事务互不干扰
- **D**urability（持久性）：事务提交后，数据永久保存

Q5：为什么要用Lombok?

答:

Lombok：一个Java库，通过注解自动生成代码

不用Lombok（传统方式）：

```
public class User {
    private Long id;
    private String username;
    private String password;

    // 需要手写getter和setter（每个字段2个方法）
    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getUsername() {
        return username;
    }

    public void setUsername(String username) {
        this.username = username;
    }

    // ... 还有很多getter/setter

    // 还要写toString、equals、hashCode等方法
}
```

用Lombok（简洁）：

```
@Data  // 一个注解搞定！
public class User {
    private Long id;
    private String username;
    private String password;
}
```

`@Data` 注解会自动生成：

- 所有字段的getter和setter
- toString()方法
- equals()和hashCode()方法

其他常用注解：

- `@Getter`：只生成getter
- `@Setter`：只生成setter
- `@NoArgsConstructor`：生成无参构造函数
- `@AllArgsConstructor`：生成全参构造函数

Q6：什么是Lambda表达式？

答：

Lambda表达式是Java 8引入的新特性，用于简化代码

传统方式（匿名内部类）：

```
// 筛选年龄大于18的用户
List<User> adults = new ArrayList<>();
for (User user : users) {
    if (user.getAge() > 18) {
        adults.add(user);
    }
}
```

Lambda方式（简洁）：

```
List<User> adults = users.stream()
    .filter(user -> user.getAge() > 18)  // Lambda表达式
    .collect(Collectors.toList());
```

语法：

```
(参数) -> { 方法体 }

// 单个参数，单行语句，可以简化为：
参数 -> 语句
```

我们项目中的例子：

```
// 筛选状态为正常的设备实例
List<DeviceInstance> normalInstances = instances.stream()
    .filter(inst -> "NORMAL".equals(inst.getStatus()))  // Lambda
    .collect(Collectors.toList());

// 将设备列表转换为VO列表
List<DeviceVO> voList = devices.stream()
    .map(device -> {  // Lambda
        DeviceVO vo = new DeviceVO();
        BeanUtils.copyProperties(device, vo);
        return vo;
    })
    .collect(Collectors.toList());
```

Q7：为什么要分DTO、VO、Entity?

答：

这三种对象虽然看起来很像，但用途不同：

Entity（实体类）：

- 对应数据库表
- 包含数据库的所有字段
- 用于MyBatis操作数据库

```
@TableName("users")
public class User {
    private Long id;
    private String username;
    private String password;  // 包含密码字段
    private String realName;
    private String role;
    private String status;
    private LocalDateTime createdAt;
    private LocalDateTime updatedAt;
}
```

DTO（Data Transfer Object，数据传输对象）：

- 用于接收前端传来的数据
- 只包含需要的字段
- 可以添加校验注解

```java
public class LoginRequest {
    @NotBlank(message = "用户名不能为空")
    private String username;

    @NotBlank(message = "密码不能为空")
    private String password;

    // 不包含其他字段，只要用户名和密码
}
```

VO（View Object，视图对象）：

- 用于返回给前端的数据
- 可以组合多个表的数据
- 不包含敏感信息（比如密码）

```java
public class UserInfo {
    private Long id;
    private String username;
    private String realName;
    private String role;

    // 不包含password字段，保护隐私
}
```

为什么要分开?

1. **安全性**：不把密码等敏感信息返回给前端
2. **灵活性**：前端需要的数据和数据库结构可能不一样
3. **解耦**：数据库表结构改变，不影响API接口

Q8：什么是MyBatis的动态SQL?

答:

动态SQL：根据条件动态生成SQL语句

场景：用户搜索功能

- 如果用户输入了用户名，就按用户名搜索
- 如果用户选择了角色，就按角色筛选
- 如果都没输入，就查询所有

传统方式（写多个SQL）：

```
// 需要写很多if-else
if (username != null && role != null) {
    sql = "SELECT * FROM users WHERE username LIKE ? AND role = ?";
} else if (username != null) {
    sql = "SELECT * FROM users WHERE username LIKE ?";
} else if (role != null) {
    sql = "SELECT * FROM users WHERE role = ?";
} else {
    sql = "SELECT * FROM users";
}
```

MyBatis动态SQL（一个SQL搞定）：

```
<select id="selectUserList" resultType="User">
    SELECT * FROM users
    <where>
        <if test="username != null and username != ''">
            AND username LIKE CONCAT('%', #{username}, '%')
        </if>
        <if test="role != null and role != ''">
            AND role = #{role}
        </if>
    </where>
</select>
```

执行效果：

```
// 情况1：username="admin", role=null
// 生成SQL：SELECT * FROM users WHERE username LIKE '%admin%'

// 情况2：username=null, role="ADMIN"
// 生成SQL：SELECT * FROM users WHERE role = 'ADMIN'

// 情况3：username="admin", role="ADMIN"
// 生成SQL：SELECT * FROM users WHERE username LIKE '%admin%' AND role = 'ADMIN'

// 情况4：username=null, role=null
// 生成SQL：SELECT * FROM users
```

Q9：前端的localStorage是什么?

答：

localStorage：浏览器提供的本地存储功能，用于保存数据

特点：

- 数据保存在浏览器中
- 关闭浏览器后数据仍然存在
- 只能存储字符串
- 容量约5MB

常用方法：

```
// 保存数据
localStorage.setItem('token', 'eyJhbGci...');
localStorage.setItem('userInfo', JSON.stringify({id: 1, username: 'admin'}));

// 读取数据
const token = localStorage.getItem('token');
const userInfo = JSON.parse(localStorage.getItem('userInfo'));

// 删除数据
localStorage.removeItem('token');

// 清空所有数据
localStorage.clear();
```

我们项目中的用途：

1. 保存JWT Token（用于后续请求认证）
2. 保存用户信息（用于显示用户名、判断角色）

安全性注意：

- localStorage可以被JavaScript访问
- 不要存储非常敏感的信息
- Token有过期时间，定期更新

Q10：如何调试代码?

答：

后端调试（IntelliJ IDEA）

方法1：打断点

1. 在代码行号左侧点击，出现红点（断点）
2. 以Debug模式运行项目
3. 当代码执行到断点时，会暂停
4. 可以查看变量的值、单步执行

方法2：查看日志

```
// 在代码中添加日志
private static final Logger logger =
LoggerFactory.getLogger(UserServiceImpl.class);

public User getUserById(Long id) {
    logger.info("查询用户，ID：{}", id);  // 打印日志
    User user = userMapper.selectById(id);
    logger.info("查询结果：{}", user);
    return user;
}
```

方法3：使用Postman测试API

1. 下载安装Postman
2. 创建请求：`GET http://localhost:9011/admin/users`
3. 添加请求头：`Authorization: Bearer <Token>`
4. 点击Send，查看响应

前端调试（浏览器）

方法1：Console控制台

```
// 在代码中添加console.log
console.log('设备列表：', devices);
console.log('响应数据：', response);

// 在浏览器按F12，打开开发者工具，切换到Console标签
// 可以看到所有的console.log输出
```

方法2：Network网络面板

1. 按F12打开开发者工具
2. 切换到Network标签
3. 刷新页面
4. 可以看到所有的HTTP请求
5. 点击某个请求，可以查看请求头、响应头、响应内容

方法3：断点调试

1. 在开发者工具的Sources标签中找到JS文件
2. 点击行号添加断点
3. 刷新页面，代码执行到断点时会暂停
4. 可以查看变量值、单步执行

Q11：如何添加新功能？

答：

示例：添加"设备维修记录"功能

步骤1：设计数据库表

```
CREATE TABLE maintenance_records (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
    device_instance_id BIGINT NOT NULL,  -- 设备实例ID
    maintenance_date DATE NOT NULL,      -- 维修日期
    maintenance_reason VARCHAR(255),     -- 维修原因
    maintenance_cost DECIMAL(10, 2),     -- 维修费用
    status VARCHAR(20) DEFAULT 'PENDING', -- 状态：PENDING/COMPLETED
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (device_instance_id) REFERENCES device_instances(id)
);
```

步骤2：创建Entity类

```
@Data
@TableName("maintenance_records")
public class MaintenanceRecord {
    @TableId(type = IdType.AUTO)
    private Long id;
    private Long deviceInstanceId;
    private LocalDate maintenanceDate;
    private String maintenanceReason;
    private BigDecimal maintenanceCost;
    private String status;
    private LocalDateTime createdAt;
}
```

步骤3：创建Mapper接口

```
public interface MaintenanceRecordMapper extends BaseMapper<MaintenanceRecord> {
    // 继承BaseMapper，自动拥有增删改查方法
}
```

步骤4：创建Service

```
@Service
public class MaintenanceServiceImpl implements MaintenanceService {
    @Autowired
    private MaintenanceRecordMapper maintenanceRecordMapper;

    @Transactional
    public MaintenanceRecord createRecord(MaintenanceRecord record) {
        // 验证设备实例是否存在
        // 创建维修记录
        maintenanceRecordMapper.insert(record);
        return record;
    }
}
```

步骤5：创建Controller

```
@RestController
@RequestMapping("/admin/maintenance")
public class MaintenanceController {
    @Autowired
    private MaintenanceService maintenanceService;

    @PostMapping
    public Result<MaintenanceRecord> createRecord(
            @RequestBody MaintenanceRecord record) {
        MaintenanceRecord created = maintenanceService.createRecord(record);
        return Result.success("创建成功", created);
    }
}
```

步骤6：创建前端页面

```
<!-- admin/maintenance.html -->
<div id="maintenanceList"></div>

<script>
async function loadRecords() {
    const response = await get('/admin/maintenance');
    renderRecords(response.data);
}
</script>
```

Q12：常见错误及解决方法

错误1：404 Not Found

原因：

- URL写错了
- Controller的@RequestMapping路径不对
- 方法的@GetMapping/@PostMapping等注解缺失

解决：

1. 检查前端请求的URL是否正确
2. 检查后端Controller的路径映射
3. 查看后端启动日志，确认接口是否注册成功

错误2：401 Unauthorized

原因：

- Token无效或过期
- 请求头中没有携带Token
- Token格式错误

解决：

1. 检查localStorage中是否有token
2. 检查请求头是否正确：`Authorization: Bearer <Token>`
3. 重新登录获取新Token

错误3：403 Forbidden

原因：

- 用户没有权限访问该接口
- 比如普通用户访问管理员接口

解决：

1. 检查用户的角色
2. 检查接口的权限要求
3. 确认是否用对了账号

错误4：500 Internal Server Error

原因：

- 后端代码出错
- 数据库连接失败
- 空指针异常等

解决：

1. 查看后端控制台的错误日志
2. 检查数据库是否启动
3. 检查配置文件中的数据库连接信息
4. 使用断点调试，定位错误位置

错误5：跨域错误（CORS）

错误信息：

```
Access to fetch at 'http://localhost:9011/devices' from origin
'http://localhost:9012'
has been blocked by CORS policy
```

原因：

- 浏览器的安全策略，阻止跨域请求
- 前端运行在9012端口，后端运行在9011端口，属于跨域

解决：

- 后端已经配置了CORS（SecurityConfig.java）
- 如果还有问题，检查配置是否生效

---

8. 学习建议

8.1 如何深入学习这个项目？

第1周：理解整体架构

- [ ] 阅读本文档，理解项目的整体结构
- [ ] 运行项目，体验所有功能
- [ ] 画出系统架构图和流程图

第2周：学习后端代码

- [ ] 从Controller开始，跟踪一个完整的请求流程
- [ ] 理解Service层的业务逻辑
- [ ] 学习MyBatis的SQL映射
- [ ] 理解Spring Security的认证流程

第3周：学习前端代码

- [ ] 理解HTML结构
- [ ] 学习CSS样式，尝试修改界面
- [ ] 理解JavaScript的异步请求
- [ ] 学习如何动态生成HTML

第4周：尝试修改和扩展

- [ ] 修改界面颜色和布局
- [ ] 添加新的筛选条件
- [ ] 尝试添加新功能（比如设备维修记录）

8.2 推荐学习资源

Spring Boot

- 官方文档：https://spring.io/projects/spring-boot
- 视频教程：B站搜索"Spring Boot教程"

MyBatis Plus

- 官方文档：https://baomidou.com/
- 特点：比MyBatis更简单，自动生成CRUD

Spring Security

- 官方文档：https://spring.io/projects/spring-security
- 重点学习：认证、授权、JWT集成

前端基础

- MDN Web文档：https://developer.mozilla.org/zh-CN/
- JavaScript教程：https://zh.javascript.info/

8.3 还是有问题，如何问我？

好的提问方式：

❌ 不好的提问：
"代码报错了，怎么办？"

✅ 好的提问：
"我在运行项目时遇到了以下错误：
错误信息：java.lang.NullPointerException at UserServiceImpl.java:45
我的操作步骤：1. 启动后端 2. 访问/admin/users接口
我的环境：JDK 17, Spring Boot 3.5.8, MySQL 8.0
我尝试过：检查了数据库连接，确认数据存在
请问可能是什么原因？"

提问时应该包含：

1. 错误信息（完整的错误堆栈）
2. 操作步骤（如何复现错误）
3. 环境信息（JDK版本、数据库版本等）
4. 已尝试的解决方法

---

9. 附录：关键代码注释版

9.1 完整的登录流程代码（带详细注释）

前端代码：

```
// 文件：login.js

// 监听表单提交事件
document.getElementById('loginForm').addEventListener('submit', async function(e)
{
    e.preventDefault();  // 阻止表单默认提交行为（防止页面刷新）

    // 获取用户输入的值
    const username = document.getElementById('username').value.trim();  // trim()
去除首尾空格
    const password = document.getElementById('password').value;

    // 前端验证：检查是否为空
    if (!username || !password) {
        alert('请输入用户名和密码');
        return;  // 终止执行
    }

    // 获取登录按钮，准备显示加载状态
    const loginBtn = document.getElementById('loginBtn');
    loginBtn.disabled = true;  // 禁用按钮，防止重复点击
    loginBtn.innerHTML = '<span class="loading"></span> 登录中...';  // 显示加载动画

    try {
        // 调用登录API（异步请求）
        const response = await API.auth.login(username, password);
        // 等待请求完成，response是后端返回的JSON数据
```

```
        // 检查响应状态码
        if (response.code === 200) {
            // 登录成功

            // 保存Token到localStorage
            localStorage.setItem('token', response.data.token);

            // 保存用户信息到localStorage（转换成JSON字符串）
            localStorage.setItem('userInfo',
JSON.stringify(response.data.userInfo));

            // 根据用户角色跳转到不同页面
            if (response.data.userInfo.role === 'ADMIN' ||
                response.data.userInfo.role === 'ROOT') {
                // 管理员 → 跳转到管理仪表盘
                window.location.href = '/admin/dashboard.html';
            } else {
                // 普通用户 → 跳转到设备浏览页面
                window.location.href = '/user/devices.html';
            }
        } else {
            // 登录失败：显示错误信息
            alert(response.message || '登录失败');
        }
    } catch (error) {
        // 捕获异常（比如网络错误）
        alert('登录失败：' + error.message);
    } finally {
        // 无论成功还是失败，都恢复按钮状态
        loginBtn.disabled = false;
        loginBtn.innerHTML = '<span>登录</span>';
    }
});
```

后端代码：

```
// 文件：AuthServiceImpl.java

@Service
public class AuthServiceImpl implements AuthService {

    @Autowired
    private UserMapper userMapper;  // 注入Mapper，用于查询数据库

    @Autowired
    private JwtUtil jwtUtil;  // 注入JWT工具类，用于生成Token

    @Autowired
    private PasswordEncoder passwordEncoder;  // 注入密码加密器，用于验证密码

    @Override
    public LoginResponse login(LoginRequest request) {
        // 步骤1：根据用户名查询数据库
        LambdaQueryWrapper<User> queryWrapper = new LambdaQueryWrapper<>();
        queryWrapper.eq(User::getUsername, request.getUsername());
```

```
        // 相当于SQL：SELECT * FROM users WHERE username = ?

        User user = userMapper.selectOne(queryWrapper);

        // 步骤2：检查用户是否存在
        if (user == null) {
            // 用户不存在，抛出异常
            throw new BusinessException(401, "用户名或密码错误");
            // 注意：不要提示"用户不存在"，避免泄露信息
        }

        // 步骤3：验证密码
        boolean isMatch = passwordEncoder.matches(
            request.getPassword(),  // 用户输入的原始密码
            user.getPassword()      // 数据库中的加密密码
        );

        if (!isMatch) {
            // 密码错误，抛出异常
            throw new BusinessException(401, "用户名或密码错误");
        }

        // 步骤4：检查用户状态
        if (!"ACTIVE".equals(user.getStatus())) {
            // 用户被禁用，抛出异常
            throw new BusinessException(403, "用户已被禁用，请联系管理员");
        }

        // 步骤5：生成JWT Token
        String token = jwtUtil.generateToken(
            user.getId(),       // 用户ID
            user.getUsername(), // 用户名
            user.getRole()      // 角色
        );

        // 步骤6：构建用户信息（不包含密码）
        UserInfo userInfo = new UserInfo();
        userInfo.setId(user.getId());
        userInfo.setUsername(user.getUsername());
        userInfo.setRealName(user.getRealName());
        userInfo.setRole(user.getRole());

        // 步骤7：构建响应对象
        LoginResponse response = new LoginResponse();
        response.setToken(token);
        response.setUserInfo(userInfo);

        // 步骤8：记录日志
        logger.info("用户登录成功，用户名：{}，角色：{}", user.getUsername(),
user.getRole());

        // 步骤9：返回响应
        return response;
    }
}
```

9.2 完整的设备借用代码（带详细注释）

```
// 文件：BorrowServiceImpl.java

@Service
public class BorrowServiceImpl implements BorrowService {

    @Autowired
    private BorrowRecordMapper borrowRecordMapper;

    @Autowired
    private DeviceMapper deviceMapper;

    @Autowired
    private DeviceInstanceMapper deviceInstanceMapper;

    /**
     * 提交借用申请
     * @param userId 用户ID（从Token中获取）
     * @param record 借用记录（包含deviceId、purpose、expectedReturnDate）
     * @return 创建的借用记录
     */
    @Override
    @Transactional(rollbackFor = Exception.class)  // 开启事务，任何异常都回滚
    public BorrowRecord applyBorrow(Long userId, BorrowRecord record) {

        // ========== 步骤1：验证设备是否存在 ==========
        if (record.getDeviceId() == null) {
            throw new BusinessException(400, "设备ID不能为空");
        }

        Device device = deviceMapper.selectById(record.getDeviceId());
        if (device == null) {
            throw new BusinessException(404, "设备不存在");
        }

        // ========== 步骤2：验证预计归还日期 ==========
        if (record.getExpectedReturnDate() == null ||
            record.getExpectedReturnDate().isBefore(LocalDate.now())) {
            throw new BusinessException(400, "预计归还日期无效");
        }

        // ========== 步骤3：查询该设备的所有实例 ==========
        List<DeviceInstance> instances = deviceInstanceMapper
            .selectWithBorrowStatus(record.getDeviceId());
        // 这个方法会查询该设备的所有实例

        // ========== 步骤4：筛选可用的实例 ==========
        DeviceInstance availableInstance = null;

        for (DeviceInstance instance : instances) {
            // 4.1 检查实例状态是否为正常
            if (!DeviceStatus.NORMAL.getCode().equals(instance.getStatus())) {
                // 如果是维修中或已报废，跳过
```

```
                continue;
            }

            // 4.2 检查实例是否已被借用
            // 查询该实例是否有状态为PENDING或APPROVED的借用记录
            LambdaQueryWrapper<BorrowRecord> borrowWrapper = new
LambdaQueryWrapper<>();
            borrowWrapper.eq(BorrowRecord::getDeviceInstanceId, instance.getId())
                    .in(BorrowRecord::getStatus,
                        BorrowStatus.PENDING.getCode(),   // 待审批
                        BorrowStatus.APPROVED.getCode()); // 已批准（借用中）

            Long borrowCount = borrowRecordMapper.selectCount(borrowWrapper);

            if (borrowCount == 0) {
                // 没有借用记录，说明可用
                availableInstance = instance;
                break;  // 找到第一个可用实例就停止
            }
        }

        // ========== 步骤5：如果没有可用实例，抛出异常 ==========
        if (availableInstance == null) {
            throw new BusinessException(400, "该设备暂无可用实例");
        }

        // ========== 步骤6：创建借用记录 ==========
        record.setDeviceInstanceId(availableInstance.getId());  // 设置实例ID
        record.setQuantity(1);  // 数量固定为1（因为是实例模式）
        record.setUserId(userId);  // 设置借用人ID
        record.setStatus(BorrowStatus.PENDING.getCode());  // 状态：待审批

        // 插入数据库
        borrowRecordMapper.insert(record);

        // ========== 步骤7：记录日志 ==========
        logger.info("提交借用申请成功，记录ID：{}，用户ID：{}，设备ID：{}，设备实例ID：
{}",
                record.getId(), userId, record.getDeviceId(),
record.getDeviceInstanceId());

        // ========== 步骤8：返回创建的记录 ==========
        return record;
    }
}
```

10. 总结

10.1 项目核心知识点

后端核心：

1. ✅ Spring Boot框架的使用
2. ✅ MyBatis Plus的数据库操作
3. ✅ Spring Security + JWT认证
4. ✅ RESTful API设计
5. ✅ 事务管理
6. ✅ 异常处理
7. ✅ 分层架构（Controller-Service-Mapper）

前端核心：

1. ✅ HTML页面结构
2. ✅ CSS样式设计（毛玻璃效果、响应式）
3. ✅ JavaScript异步请求（Fetch API）
4. ✅ DOM操作（动态生成HTML）
5. ✅ localStorage存储
6. ✅ 事件处理

数据库核心：

1. ✅ 表设计（5张表）
2. ✅ 外键约束
3. ✅ 索引优化
4. ✅ 设备模板+实例模式

10.2 学习路径建议

```
第1阶段：理解项目（1-2周）
    ↓
    - 运行项目，体验功能
    - 阅读本文档，理解架构
    - 画流程图和架构图
    ↓
第2阶段：学习后端（2-3周）
    ↓
    - 学习Spring Boot基础
    - 理解依赖注入、注解
    - 学习MyBatis Plus
    - 理解JWT认证流程
    ↓
第3阶段：学习前端（1-2周）
    ↓
    - 复习HTML/CSS/JavaScript
    - 学习Fetch API
```

```
    - 理解异步编程
    - 学习DOM操作
    ↓
第4阶段：实践扩展（持续）
    ↓
    - 修改现有功能
    - 添加新功能
    - 优化代码
    - 学习新技术
```

10.3 最后的话

这个项目虽然功能完整，但还有很多可以改进和扩展的地方。希望大家通过学习这个项目，能够：

1. **理解前后端分离架构**：这是目前主流的开发模式
2. **掌握Spring Boot开发**：企业级Java开发的必备技能
3. **学会数据库设计**：合理的数据库设计是系统的基础
4. **提升代码质量意识**：规范的代码更易维护

记住：

- 遇到问题不要慌，先看错误信息，再查日志
- 多动手实践，多写代码
- 不懂就问，不要憋着
- 学习是一个循序渐进的过程

加油！ 💪

附录A：常用命令速查

Maven命令

```bash
mvn clean           # 清理编译文件
mvn compile         # 编译项目
mvn package         # 打包项目
mvn spring-boot:run  # 运行Spring Boot项目
```

Git命令

```bash
git status          # 查看状态
git add .           # 添加所有文件
git commit -m "提交信息"  # 提交
git push            # 推送到远程仓库
git pull            # 拉取远程代码
```

MySQL命令

```
SHOW DATABASES;     -- 查看所有数据库
USE device_manage; -- 切换数据库
SHOW TABLES;        -- 查看所有表
DESC users;         -- 查看表结构
SELECT * FROM users;  -- 查询数据
```

附录B：项目文件清单

后端核心文件

```
DeviceManage-backend/src/main/java/chat/hbcit/devicemanagebackend/
├── DeviceManageBackendApplication.java  ← 启动类
├── config/
│   ├── SecurityConfig.java              ← 安全配置（JWT、CORS）
│   ├── MyBatisPlusConfig.java           ← MyBatis配置
│   └── JacksonConfig.java               ← JSON序列化配置
├── controller/
│   ├── AuthController.java              ← 登录接口
│   ├── DeviceController.java            ← 设备浏览接口（用户）
│   ├── BorrowController.java            ← 借用申请接口（用户）
│   ├── AdminDeviceController.java       ← 设备管理接口（管理员）
│   ├── AdminBorrowController.java       ← 借用审批接口（管理员）
│   └── AdminUserController.java         ← 用户管理接口（管理员）
├── service/
│   ├── AuthService.java                 ← 认证服务接口
│   ├── DeviceService.java               ← 设备服务接口
│   ├── BorrowService.java               ← 借用服务接口
│   └── impl/                            ← 服务实现类
├── mapper/
│   ├── UserMapper.java                  ← 用户Mapper
│   ├── DeviceMapper.java                ← 设备Mapper
│   └── BorrowRecordMapper.java          ← 借用记录Mapper
├── entity/
│   ├── User.java                        ← 用户实体
│   ├── Device.java                      ← 设备实体
│   └── BorrowRecord.java                ← 借用记录实体
└── util/
    └── JwtUtil.java                     ← JWT工具类
```

前端核心文件

```
DeviceManage-frontend/
├── index.html                           ← 登录页面
├── user/
│   ├── devices.html                     ← 设备浏览页面
│   └── borrow-records.html              ← 我的借用记录
├── admin/
│   ├── dashboard.html                   ← 管理仪表盘
│   ├── devices.html                     ← 设备管理
│   ├── users.html                       ← 用户管理
```

```
|   └── borrow-approval.html          ← 借用审批
├── css/
|   ├── common.css                    ← 通用样式
|   ├── layout.css                    ← 布局样式
|   └── login.css                     ← 登录页样式
└── js/
    ├── api.js                        ← API请求封装
    ├── auth.js                       ← 认证相关
    ├── layout.js                     ← 布局相关
    └── login.js                      ← 登录逻辑
```

文档版本：v1.1
最后更新：2025年12月25日